Aula 11:
Geometria Computacional

***Disciplina*: Maratona de Programação I**

Profs. Edmilson Marmo e Luiz Olmes

edmarmo@unifei.edu.br, olmes@unifei.edu.br

UNIFEI
UNIVERSIDADE FEDERAL DE ITAJUBÁ

Nas aulas anteriores...

- **O QUE JÁ ESTUDAMOS?**

- Introdução à Maratona
- Problemas ad hoc
- Standard Template Library (STL)
- Grafos: DFS e BFS
- Grafos: algoritmos de Dijkstra - Tarjan
- Union-Find

Objetivos

- Reconhecer problemas clássicos de geometria computacional em competições.
- Aplicar predicados geométricos básicos para pontos, segmentos e polígonos.
- Comparar soluções diretas com algoritmos mais eficientes.
- Implementar rotinas geométricas reutilizáveis em C++.

Introdução

- Geometria Computacional estuda algoritmos eficientes para problemas envolvendo **objetos geométricos**.
- Entradas típicas: pontos, retas, segmentos e polígonos.
- Combina **geometria euclidiana**, **estruturas de dados** e **análise de algoritmos**.
- Em programação competitiva, o desafio costuma ser transformar uma propriedade geométrica em um teste computacional simples.

Introdução

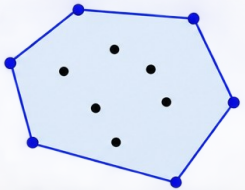
Problemas Clássicos em Geometria Computacional

Par de pontos mais próximos



Dados n pontos no plano, encontrar dois deles que estão à **distância mínima**.

Fecho convexo



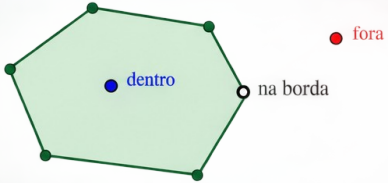
Dado um conjunto de pontos no plano, determinar o **fecho convexo**.

Interseção de segmentos



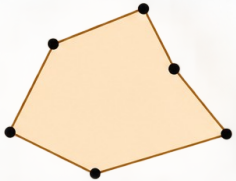
Dada uma coleção de segmentos no plano, decidir se existem dois segmentos que se **intersectam**.

Ponto em polígono



Dado um polígono e um ponto, determinar se o ponto está **dentro**, **fora** ou **na borda** do polígono.

Área de polígonos



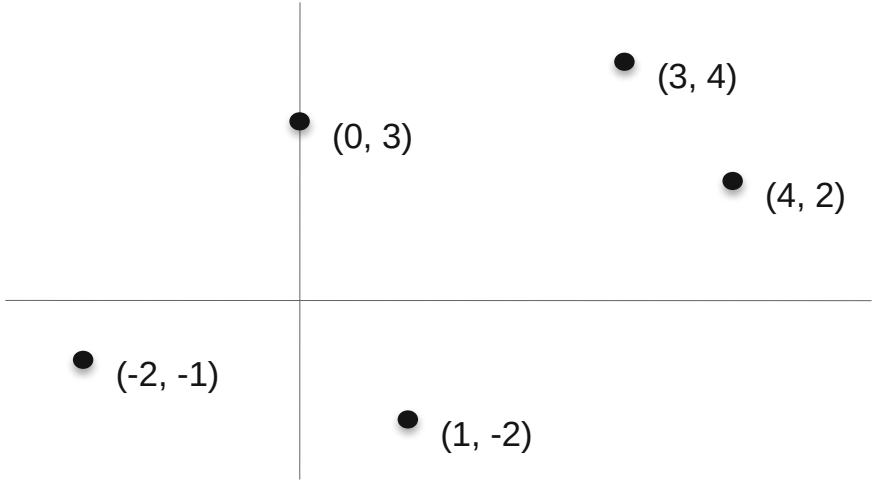
$$A = \frac{1}{2}\left|\sum_{i=1}^{n}(x_i y_{i+1} - x_{i+1} y_i)\right|$$

$$(x_{n+1}, y_{n+1}) = (x_1, y_1)$$

Dado um polígono (convexo ou não), calcular sua **área**.

Par de pontos mais próximos

- **Problema:** Dados *n* pontos no plano, determinar dois deles que estão à distância mínima.
- **Entrada:** Uma coleção de *n* pontos, representados por suas coordenadas (x,y).



x	-2	0	1	3	4
y	-1	3	-2	4	2

Par de pontos mais próximos

- Um ponto no plano é determinado por duas coordenadas:

$$p = (x, y)$$

```
struct Ponto {
    long long x, y;
};

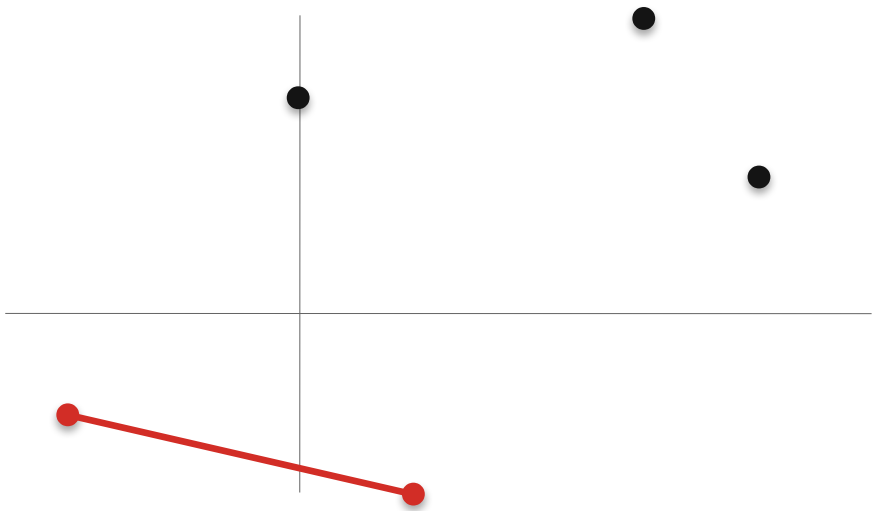
Ponto P[5] = {
    {-2, -1},
    { 0,  3},
    { 1, -2},
    { 3,  4},
    { 4,  2}
};
```



Par de pontos mais próximos

- Para comparar dois pontos, utilizamos a distância euclidiana.

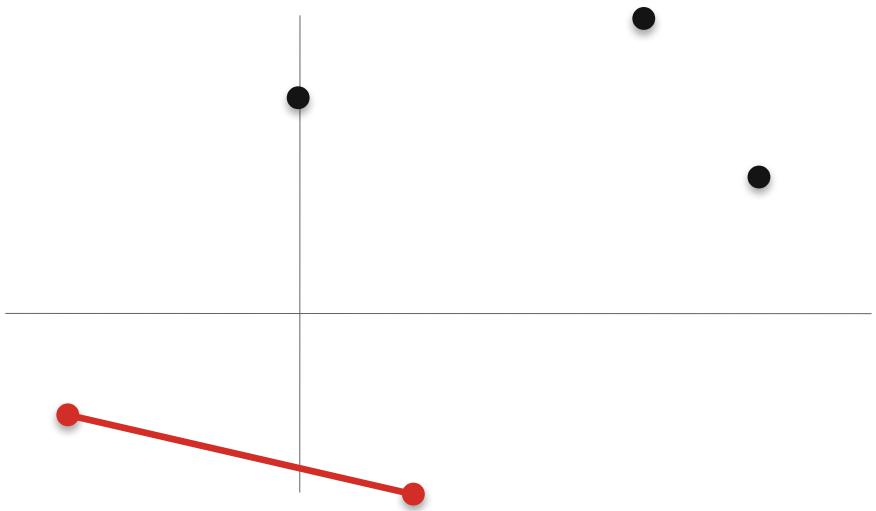
$$d(p_i,p_j) = \sqrt{((x_i - x_j)^2 + (y_i - y_j)^2)}$$



Par de pontos mais próximos

- Em competições, se o objetivo é apenas comparar distâncias, podemos evitar a raiz quadrada.

$$d^2(p_i,p_j) = (x_i - x_j)^2 + (y_i - y_j)^2$$



Par de pontos mais próximos – Algoritmo elementar

- Primeira solução: testar todos os pares de pontos.

```
long long menor = LLONG_MAX;
for (int i=0; i < n; i++)
  for (int j=i+1; j < n; j++)
     menor = min (menor, dist2(P[i],P[j]));
```

Número de pares

$$n(n-1)/2$$

Complexidade

$$O(n^2)$$

Par mais próximo na reta

- **Problema:** Dados *n* pontos em uma reta, determinar dois deles que estão à distância mínima.



Par mais próximo na reta

- **Problema:** Dados *n* pontos em uma reta, determinar dois deles que estão à distância mínima.



- Ordenando os pontos, o par mais próximo necessariamente será formado por pontos consecutivos.
- **Ordenação:** O(*n* log *n*) | Varredura dos consecutivos: O(*n*)

Complexidade: O(*n* log *n*)

Par mais próximo na reta

- **Problema:** Dados *n* pontos em uma re[illegible] determ[illegible] distância mínima.



- Ordenando os pontos, o par mais próximo neces[illegible] pontos consecutivos.
- **Ordenação:** O(*n* log *n*) | Varredura dos consecutivos: O(*n*)

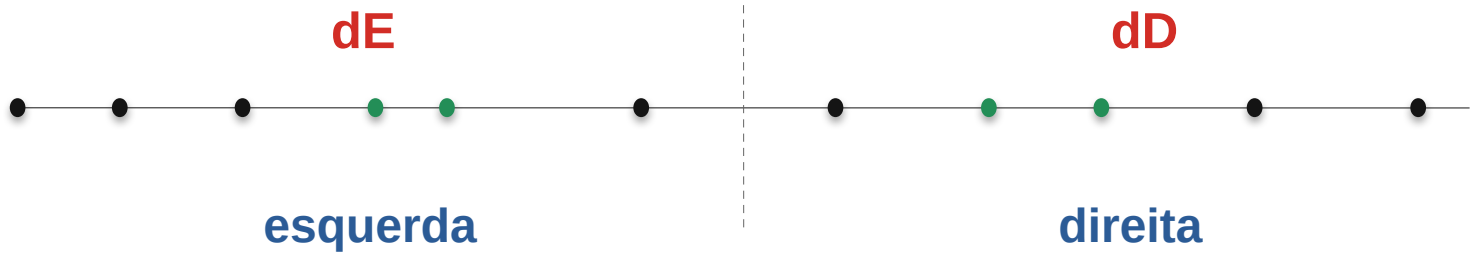
Problema:

Como generalizar esta solução para o plano?

Complexidade: O(*n* log *n*)

Divisão e conquista

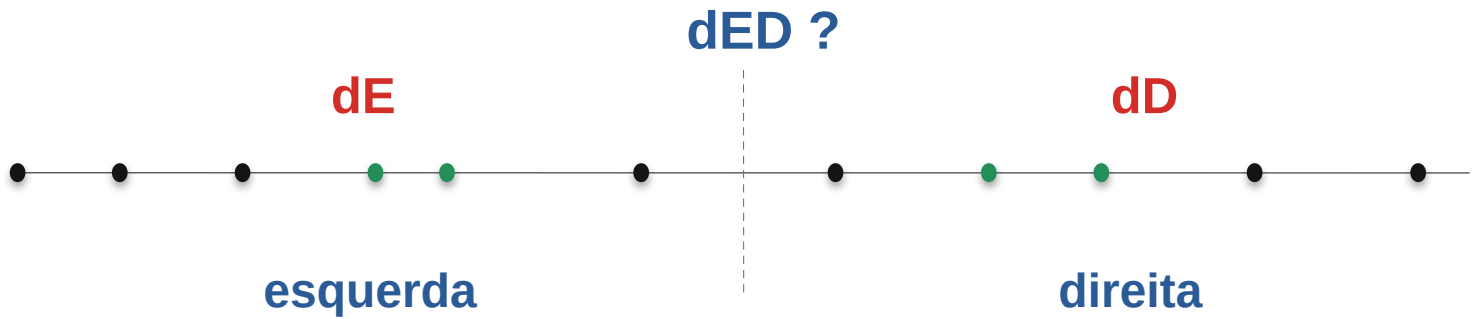
- **Divisão:** dividir a instância em duas partes menores.
- **Conquista:** resolver recursivamente cada uma das partes.
- **Combinação:** combinar as duas soluções para obter a solução do problema original.



Divisão e conquista

- Depois de resolver as duas metades, ainda falta verificar se o par mais próximo cruza a divisão.

$$d = \min(dE, dD)$$



- A etapa de **combinação** é a chave para obter uma solução eficiente no plano.

Par mais próximo na reta – Código C++

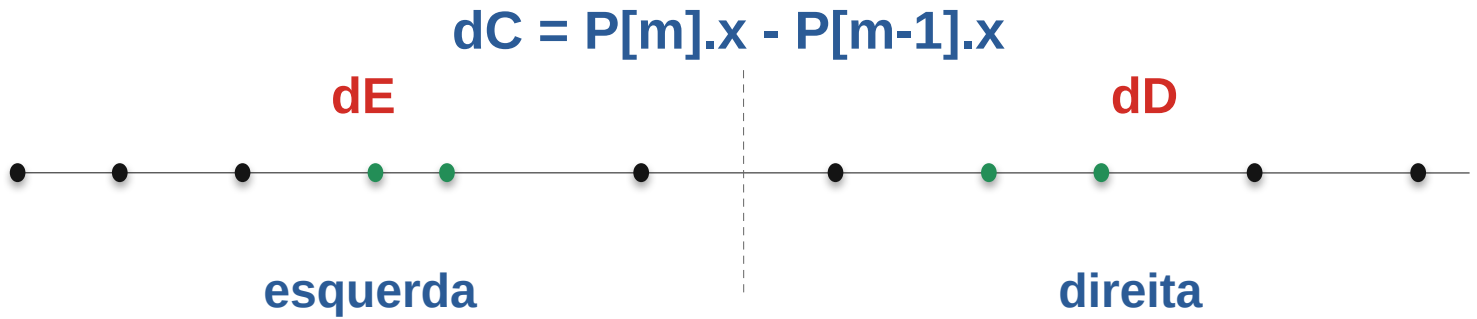
- Considerando *P* ordenado por *x* e intervalos [*ini*, *fim*):

```
long long combina(Ponto P[], int meio,
                  long long dE, long long dD) {
    long long dC = P[meio].x - P[meio - 1].x;
    return min(min(dE, dD), dC);
}
```

Par mais próximo na reta - **Combinação**

- Após resolver as duas metades, deve-se comparar os pontos da fronteira.

d = min(dE, dD, P[m].x - P[m-1].x)



Par mais próximo na reta – Código C++

- Pré-condição: pontos ordenados por *x*.

```
long long maisProximoReta(Ponto P[], int ini, int fim) {

    if (fim - ini == 2)
        return P[ini + 1].x - P[ini].x;

    if (fim - ini == 1)
        return LLONG_MAX;

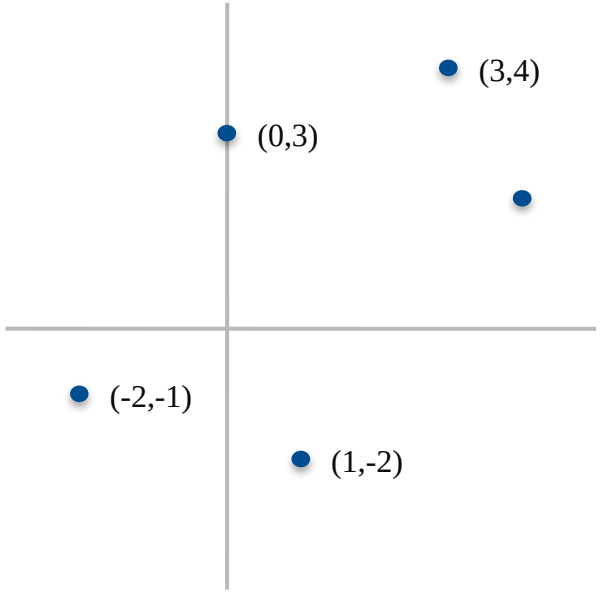
    int meio = (ini + fim) / 2;

    long long dE = maisProximoReta(P, ini, meio);
    long long dD = maisProximoReta(P, meio, fim);

    return combina(P, meio, dE, dD);
}
```

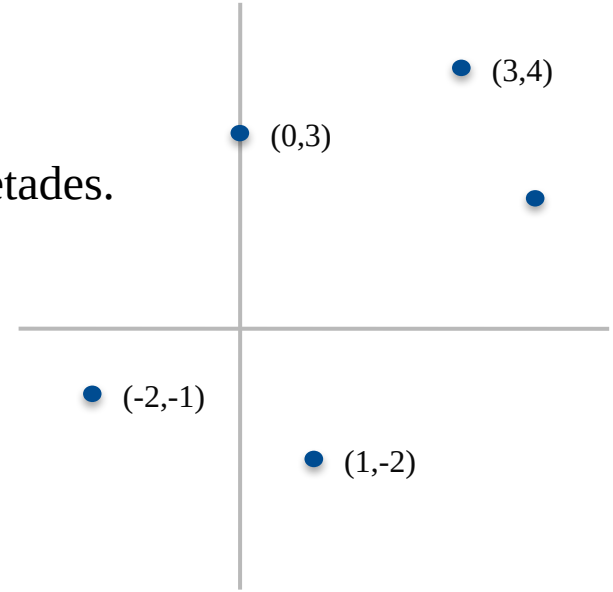
Par mais próximo no plano

- Obtivemos um algoritmo O(n lg n) para pontos na reta.
- Como generalizar essa ideia para o plano?



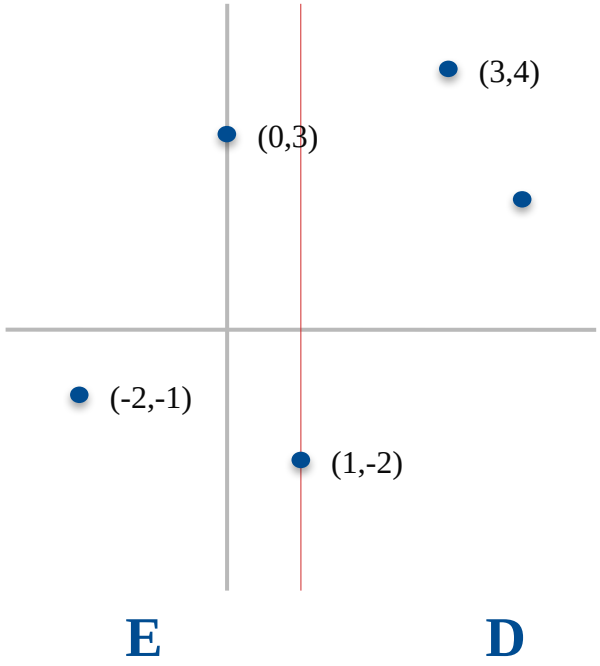
Par mais próximo no plano

- **Ordenar** os pontos pela coordenada *x*.
- **Dividir** o conjunto em duas metades, E e D.
- **Resolver** recursivamente cada metade.
- **Combinar** os resultados, verificando pares entre as metades.



Par mais próximo no plano

- A reta vertical separa os pontos em duas partes de tamanhos
- aproximadamente iguais.

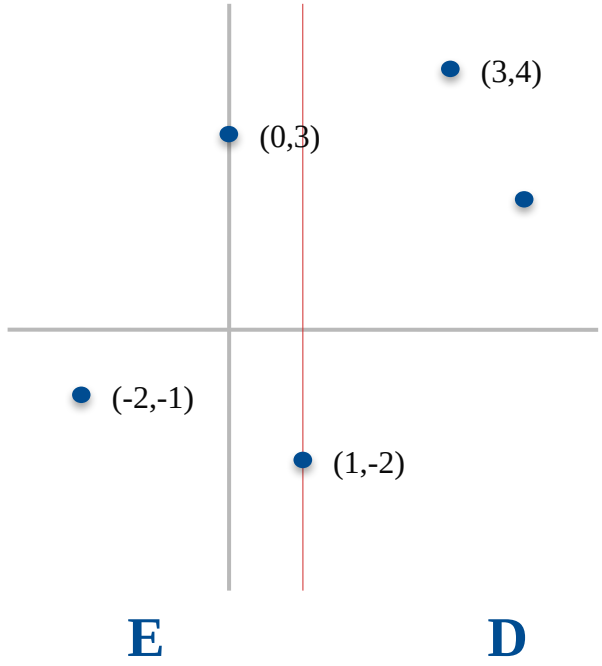


Par mais próximo no plano

- Calcular, recursivamente, as menores distâncias:

dE — menor distância em E
dD — menor distância em D

d = min(dE, dD)



Par mais próximo no plano

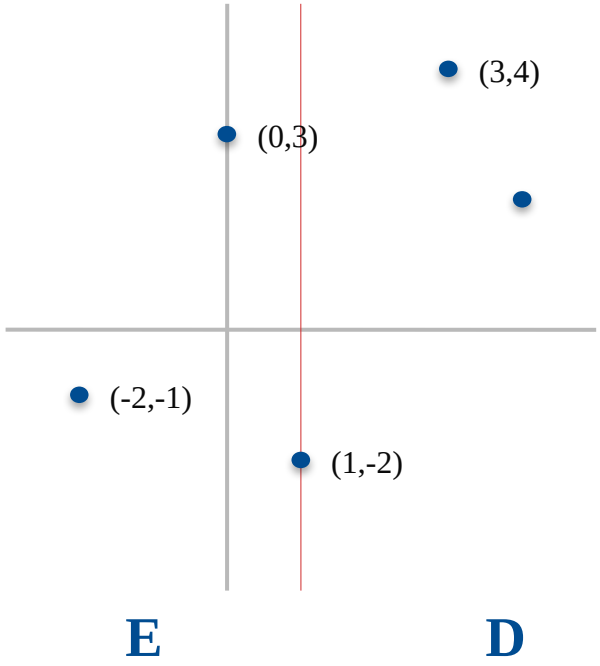
- Calcular, recursivamente, as menores distâncias:

dE — menor distância em E
dD — menor distância em D

d = min(dE, dD)

Problema:

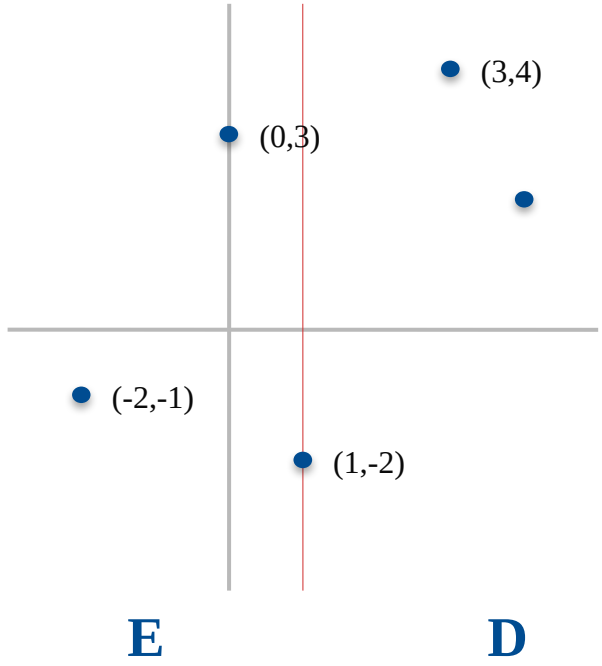
Como Combinar ?



Par mais próximo no plano

- O par mais próximo pode ter um ponto em E e outro em D.
- **Por isso, não basta retornar min(dE, dD).**
- Basta procurar pares que possam melhorar d.

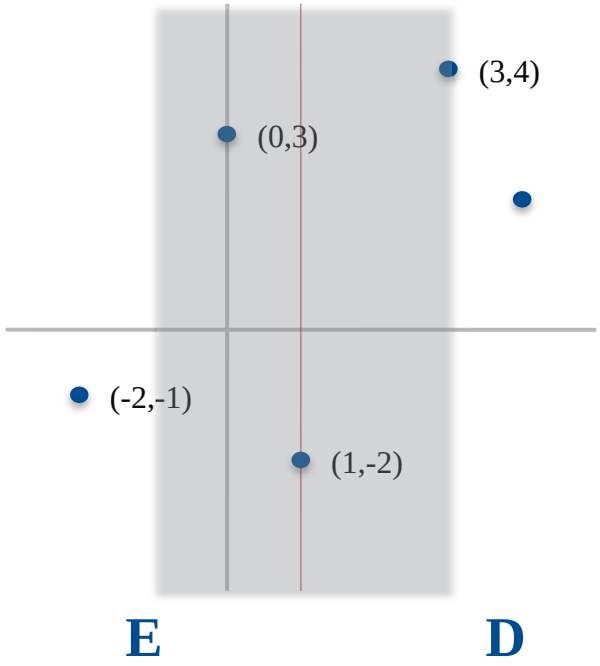
$$d = \min(dE, dD)$$



Par mais próximo no plano

- Se dois pontos de lados opostos estão a uma
- distância menor que *d*, ambos estão a menos
- de *d* da reta divisória.

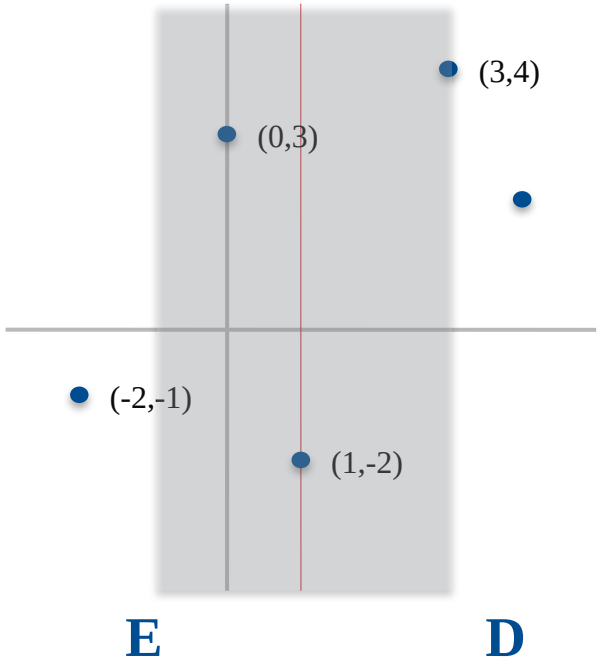
- **Logo, basta examinar os pontos na faixa central.**



Par mais próximo no plano

- Os pontos da faixa são examinados em ordem
- crescente da coordenada *y*.

- **Para cada ponto, só interessam os seguintes cuja diferença em *y* seja menor que *d*.**



Por que poucos candidatos?

- Após resolver as duas metades, seja
- $d = \min(dE, dD)$.
- Os pontos da faixa são ordenados por y.
- Para cada ponto p, só podem melhorar d
- os pontos q com $0 \leq y(q) - y(p) < d$.
- Esses candidatos ficam em um retângulo
- de largura $2d$ e altura d.



Argumento geométrico de empacotamento

- Como *d* é a menor distância encontrada em
- cada metade, dois pontos da mesma metade
- **não** podem estar a uma distância menor que
- *d*.
- Divide-se cada metade do retângulo em 4
- quadrados de lado *d*/2.

Diagonal = $\sqrt{[(d/2)^2 + (d/2)^2]} = d/\sqrt{2} < d$

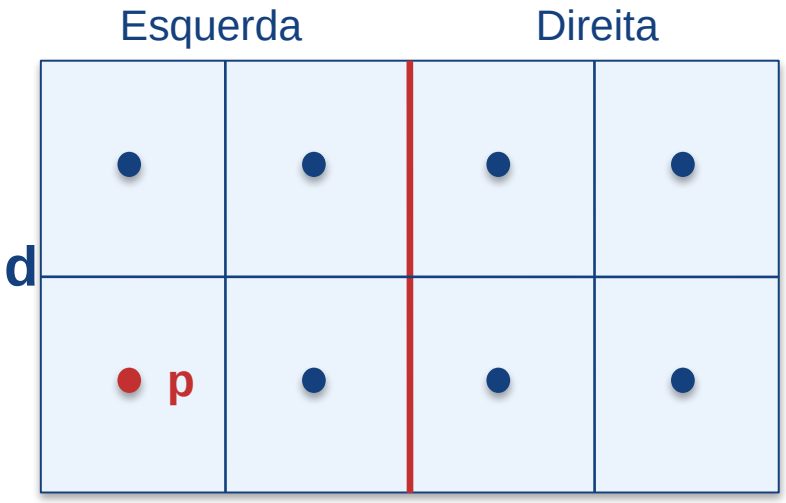
Portanto, cada quadrado contém no máximo um ponto.



4 quadrados em cada lado = 8 no total

Por que poucos candidatos?

- Há 8 quadrados na região de candidatos.
- Cada quadrado pode conter no máximo 1
- ponto.
- Assim, a região contém no máximo 8
- pontos, incluindo *p*.
- Logo, **basta comparar** *p* com até 7 pontos
- seguintes na ordenação de *y*.



Consequência: a combinação custa O(n), desde que a ordenação por y seja mantida.

Limite constante de comparações por ponto

Algoritmo – visão geral

ParMaisProximo (P)

1. Se há poucos pontos, resolver diretamente.
2. **Dividir** *P* em duas metades pela coordenada *x*.
3. **Calcular** *dE* e *dD* recursivamente.
4. **Fazer** $d = \min(dE, dD)$.
5. **Construir** a faixa $|x - x_{meio}| < d$. (a condição é verificada por $(x - x_{meio})^2 < d^2$)
6. **Examinar** candidatos próximos em ordem de y.
7. **Retornar** a menor distância encontrada.

Exemplo de implementação

- Coordenadas inteiras: as comparações utilizam a distância do quadrado.
- Usar long long exige que os limites das coordenadas evitem overflow.

```
#include <bits/stdc++.h>
using namespace std;

struct Ponto {
    long long x, y;
};

long long dist2(const Ponto& a, const Ponto& b) {
    long long dx = a.x - b.x;
    long long dy = a.y - b.y;
    return dx * dx + dy * dy;
}
```

Exemplo de implementação

```
#include <bits/stdc++.h>
using namespace std;

bool porX(const Ponto& a, const Ponto& b) {
    if (a.x != b.x)
       return a.x < b.x;
    return a.y < b.y;
}

bool porY(const Ponto& a, const Ponto& b) {
    if (a.y != b.y)
       return a.y < b.y;
    return a.x < b.x;
}

// No início do programa:
// sort(P.begin(), P.end(), porX);
```

Exemplo de implementação

```
long long resolver(vector<Ponto>& P, int ini, int fim,
                   vector<Ponto>& aux) {
    int n = fim - ini;
    if (n <= 3) {
        long long d = LLONG_MAX;
        for (int i = ini; i < fim; ++i)
            for (int j = i + 1; j < fim; ++j)
                d = min(d, dist2(P[i], P[j]));
        sort(P.begin() + ini, P.begin() + fim, porY);
        return d;
    }
    int meio = (ini + fim) / 2;
    long long xMeio = P[meio].x;
```

Exemplo de implementação

```
long long dE = resolver(P, ini, meio, aux);
long long dD = resolver(P, meio, fim, aux);
long long d = min(dE, dD);

merge(P.begin() + ini, P.begin() + meio,
      P.begin() + meio, P.begin() + fim,
      aux.begin() + ini, porY);
copy(aux.begin() + ini, aux.begin() + fim,
     P.begin() + ini);
```

Exemplo de implementação

```
vector<Ponto> faixa;
for (int i = ini; i < fim; ++i) {
    long long dx = P[i].x - xMeio;
    if (dx * dx < d)
        faixa.push_back(P[i]);
}

// A faixa já está ordenada por y.
// d guarda a menor distância ao quadrado.
```

Exemplo de implementação

```
    for (int i = 0; i < (int)faixa.size(); ++i) {
        for (int j = i + 1;
             j < (int)faixa.size() && j <= i + 7; ++j) {
            long long dy = faixa[j].y - faixa[i].y;
            if (dy * dy >= d)
               break;
            d = min(d, dist2(faixa[i], faixa[j]));
        }
    }
    return d;
}
```

Exemplo de implementação

```
int main() {
    vector<Ponto> P = {
        {-2, -1}, {0, 3}, {1, -2},
        {3, 4}, {4, 2}
    };
    if (P.size() < 2)
       return 0;

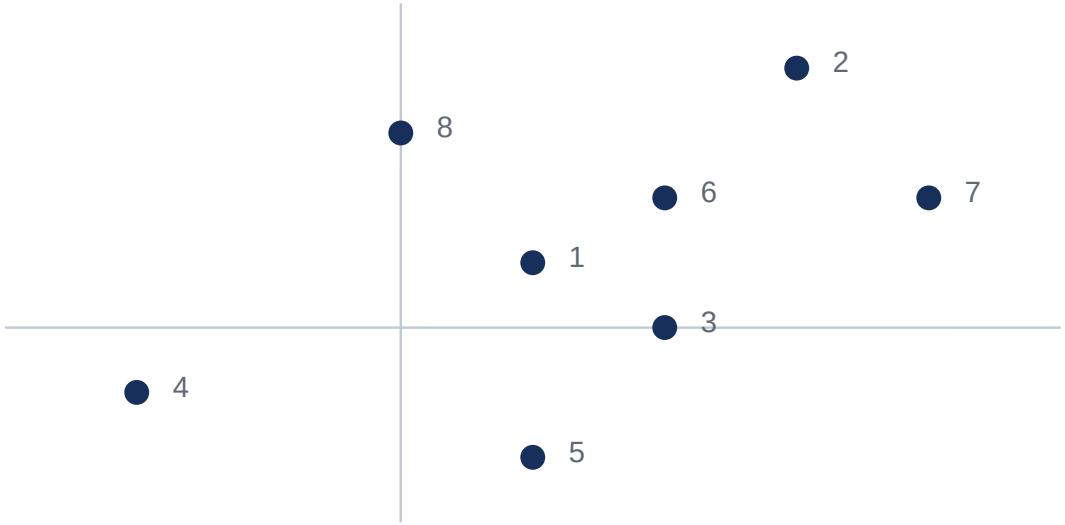
    sort(P.begin(), P.end(), porX);
    vector<Ponto> aux(P.size());
    long long d2 = resolver(P, 0, (int)P.size(), aux);
    cout << "Distancia ao quadrado: " << d2 << '\n';
    cout << "Distancia: " << sqrt((long double)d2) << '\n';
}
```

Problemas semelhantes no **beecrowd**



Fecho convexo - Motivação

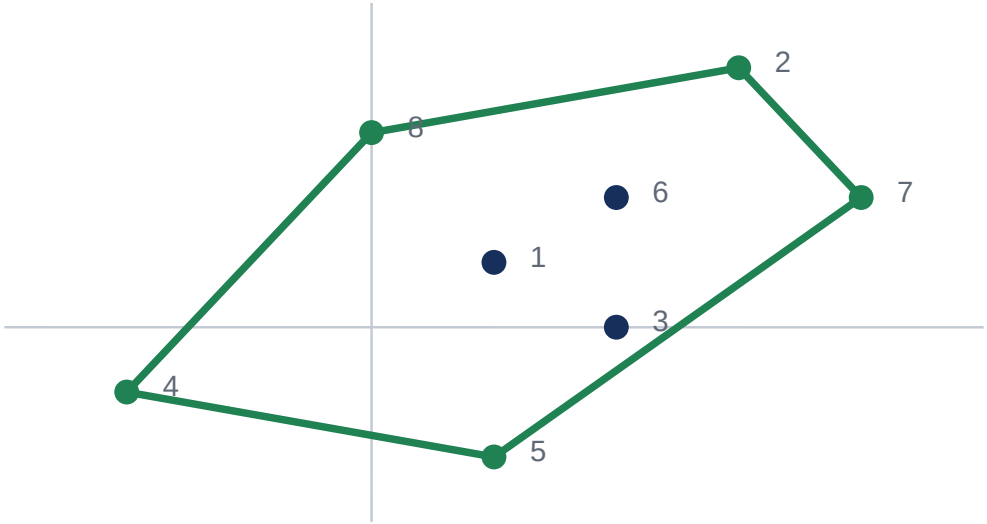
- Dado um conjunto de pontos, qual é o menor polígono convexo que contém todos eles?



Problema: determinar os pontos da fronteira.

Fecho convexo – Intuição geométrica

- Imagine um elástico envolvendo todos os pontos do plano.



O elástico delimita o fecho convexo.

Pontos internos não são vértices do polígono.

Combinação convexa de dois pontos

- Sejam p e q dois pontos do plano.
- Uma combinação convexa de p e q é um ponto r da forma:

$$r = (1 - \lambda)p + \lambda q, \quad 0 \leq \lambda \leq 1$$



Para λ = 0, r = p; para λ = 1, r = q; para 0 < λ < 1, r está entre p e q.

Combinação convexa – Exemplo

- Considere p = (0,0) e q = (4,0) e λ = 0,25.

r = 0,75p + 0,25q = (1,0)

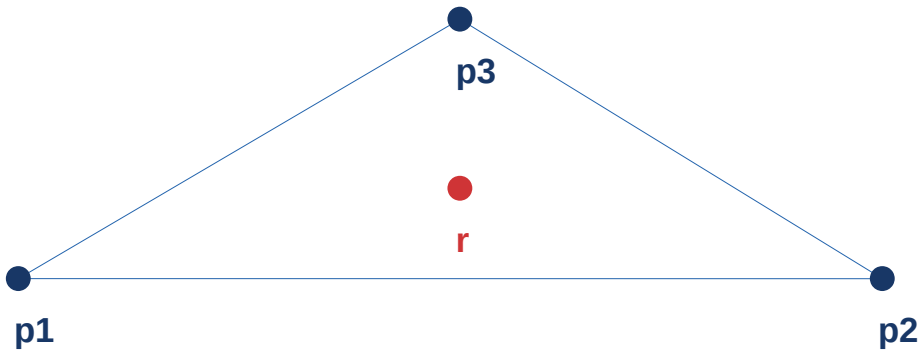


Os pesos determinam a posição de r no segmento entre p e q.

Combinação convexa de vários pontos

- Para pontos $p_1, p_2, \ldots, p_k$, uma combinação convexa é:

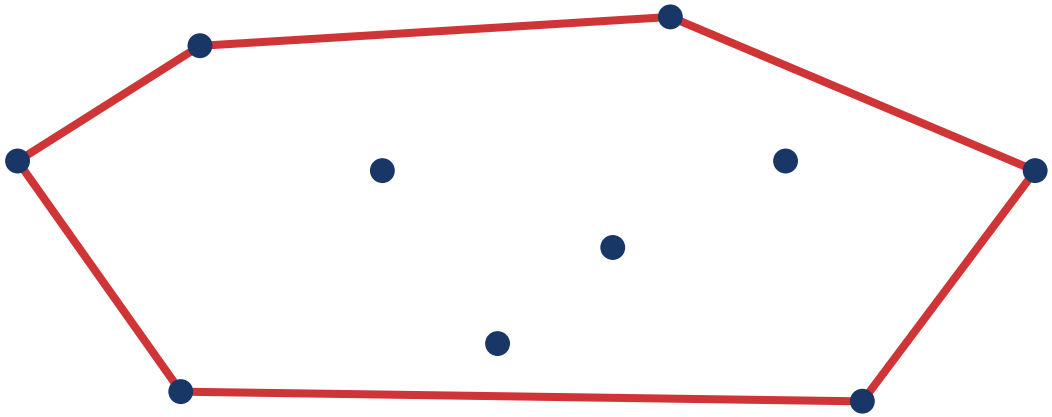
$$\mathbf{r = \lambda_1 p_1 + \lambda_2 p_2 + \cdots + \lambda_k p_k} \qquad \lambda_i \geq 0 \text{ e } \lambda_1 + \lambda_2 + \cdots + \lambda_k = 1$$



Com três pontos não colineares, r pode ocupar qualquer posição no triângulo.

Combinação convexa de vários pontos

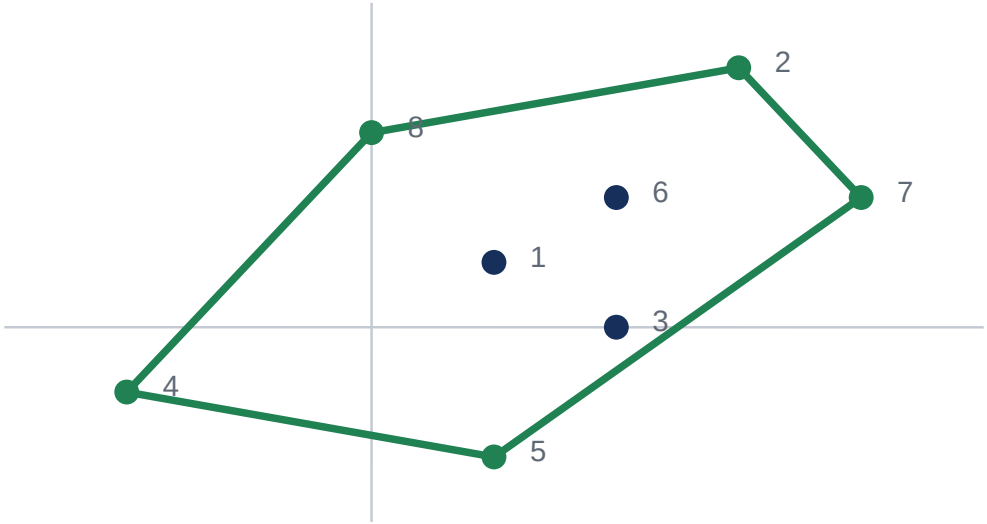
- O fecho convexo é o conjunto de todas as combinações convexas dos pontos.
- É a **menor região convexa** que contém todos eles.



Aplicação: identificar os pontos extremos que delimitam o conjunto.

Definição de fecho convexo

- `conv(P)` é o conjunto de todas as combinações convexas de pontos de P.



Em duas dimensões, a fronteira é um polígono convexo (ou um caso degenerado)

Pontos extremos - Definição

- Um ponto $p \in P$ é extremo se não pode ser escrito como combinação convexa dos demais pontos de P.

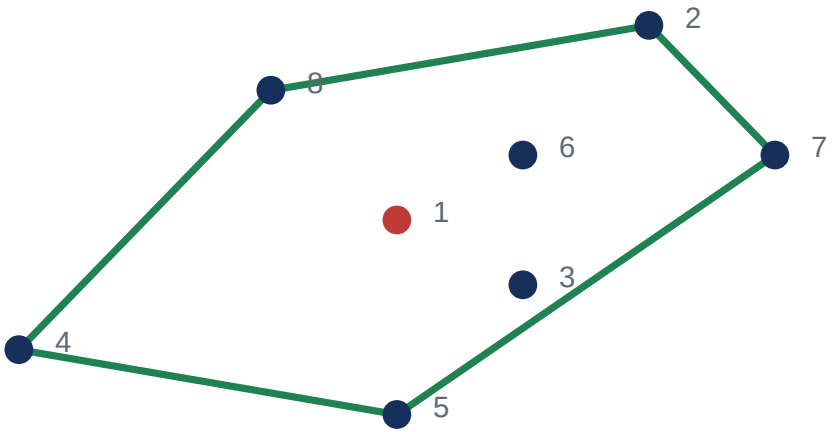
$$p \notin \text{conv}(P \setminus \{p\})$$

Se p pode ser obtido como combinação convexa dos outros pontos, então p não é extremo.

Se não pode, p é necessário para determinar o fecho.

Ponto não extremo – podemos removê-lo

- Considere o ponto 1, situado no interior do fecho convexo.



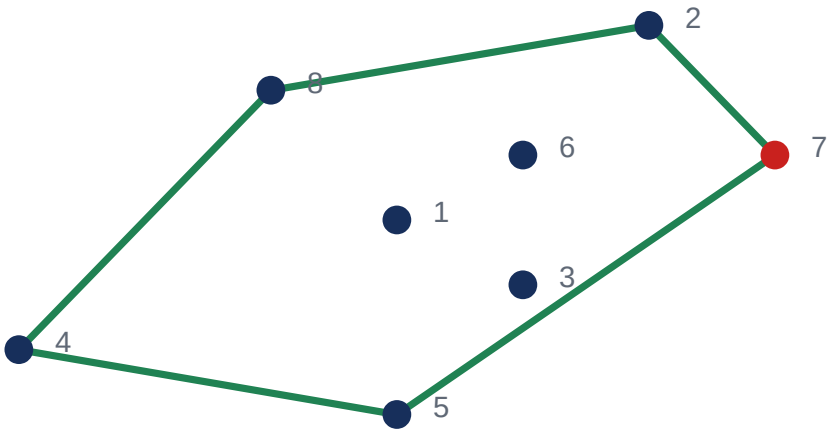
O ponto 1 pode ser obtido por uma combinação convexa dos demais.

Removê-lo não altera o fecho convexo.

conv(P) = conv(P \ {p_1})

Ponto extremo – não podemos removê-lo

- Agora considere o ponto 7, um vértice da fronteira.



O ponto 7 não é combinação convexa dos demais pontos.

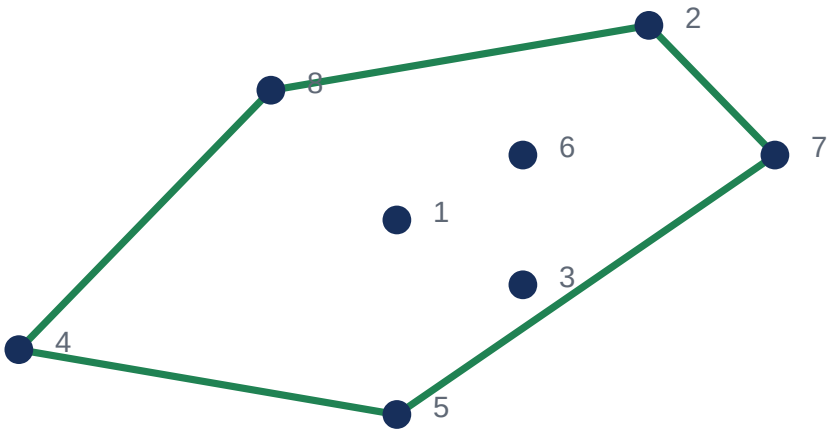
Removê-lo altera o fecho convexo.

$conv(P) \neq conv(P \setminus \{p_7\})$

O menor conjunto que determina o fecho

- Se E é o conjunto de pontos extremos de P, então:

conv(E) = conv(P)



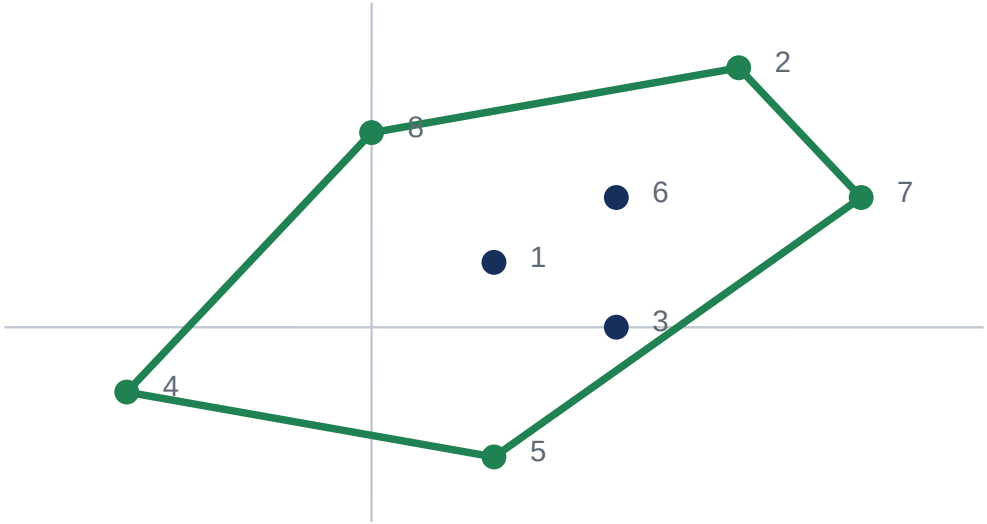
E = {2, 4, 5, 7, 8}

Esses pontos são suficientes – e nenhum deles pode ser retirado.

Os pontos extremos constituem o menor subconjunto de P com o mesmo fecho convexo.

Definição de fecho convexo

- Pontos extremos: são os vértices do fecho.

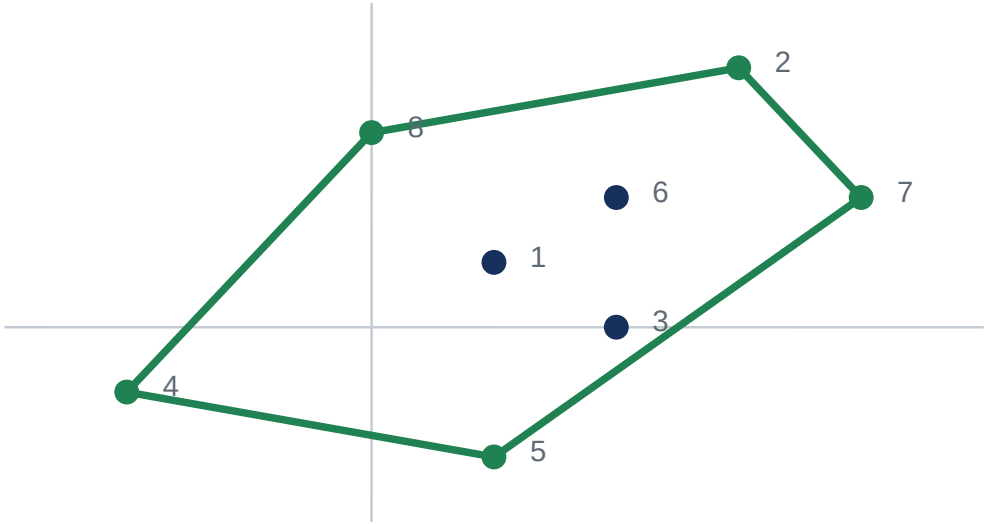


Os demais pontos podem estar no interior ou nas arestas.

No exemplo, os extremos são os pontos 2, 4, 5, 7 e 8.

Representação do fecho convexo

- Armazenamos os vértices em ordem anti-horária.



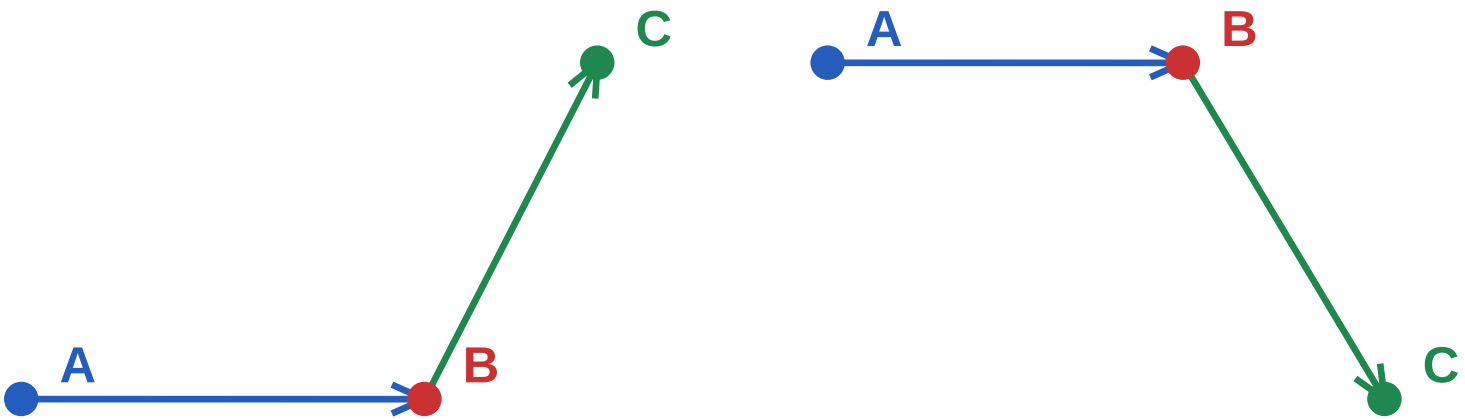
H = [4, 5, 7, 2, 8]

h = 5

A escolha do primeiro vértice é arbitrária; a ordem circular importa.

Predicados geométricos - Orientação

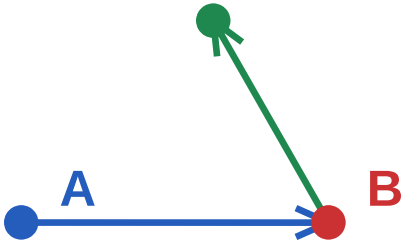
- Dados três pontos A, B e C, em que sentido fazemos a curva A → B → C?



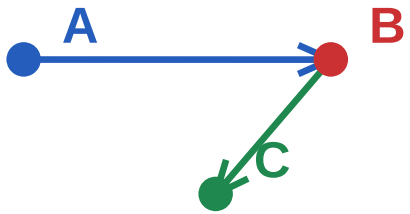
Curva à esquerda **Curva à direita**

Predicados geométricos – Orientação – 3 possibilidades

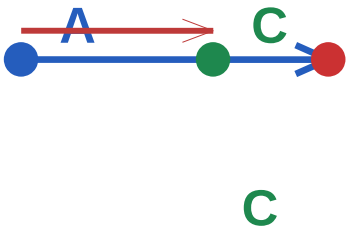
- O sentido da curva depende da posição de C em relação à reta orientada AB.



Esquerda (+)



Direita (−)



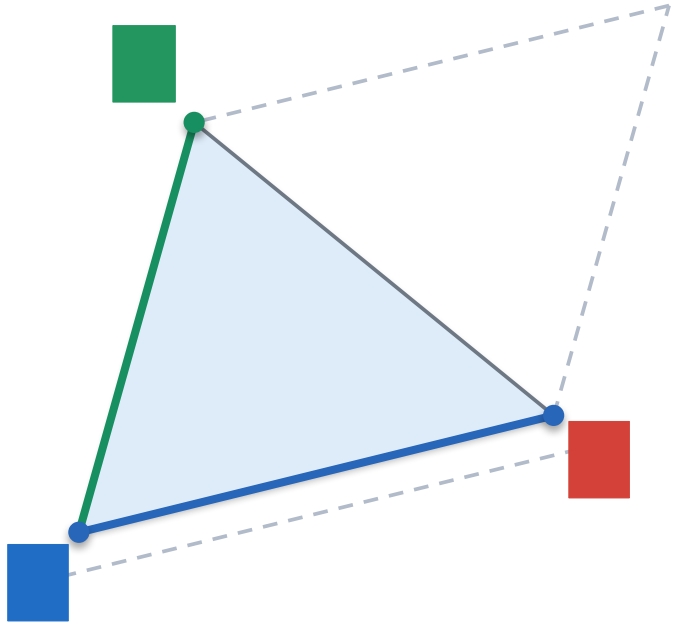
Colinear (0)

Predicados geométricos – Orientação – Determinante

- O sinal do determinante indica a orientação; seu módulo mede a área.

$$D = \begin{vmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{vmatrix}$$

|D| = 2 × área do triângulo ABC



$$D = (x_2 - x_1)(y_3 - y_1) - (y_2 - y_1)(x_3 - x_1)$$

D > 0 : esquerda
D < 0 : direita
D = 0 : colineares

Predicados geométricos – Orientação – Determinante

- O determinante fornece um teste numérico para a orientação.

orient(A,B,C) = (Bx − Ax)(Cy − Ay) − (By − Ay)(Cx − Ax)

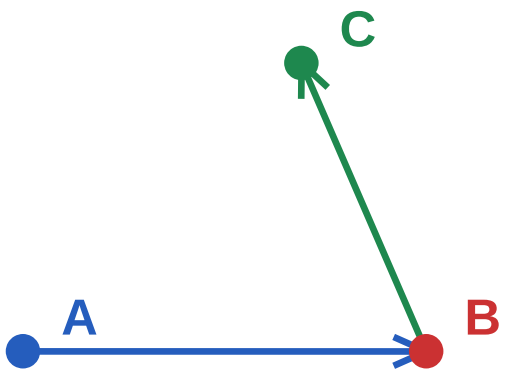
orient(A,B,C) > 0 → curva à esquerda (anti-horária)

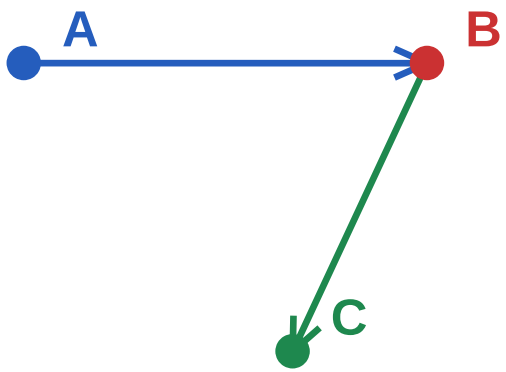
orient(A,B,C) < 0 → curva à direita (horária)

orient(A,B,C) = 0 → pontos colineares

Predicados geométricos – Orientação – Exemplo

- Considere A = (0,0), B = (3,0) e dois candidatos para C.





C = (2,2) → orient = +6

C = (2,−2) → orient = −6

Orientação – Implementação em C++

```
struct Ponto {
    long long x, y;
};


long long orientacao(Ponto a, Ponto b, Ponto c) {
    return (b.x - a.x) * (c.y - a.y)
         - (b.y - a.y) * (c.x - a.x);
}
```

Para coordenadas grandes, verificar o risco de overflow.

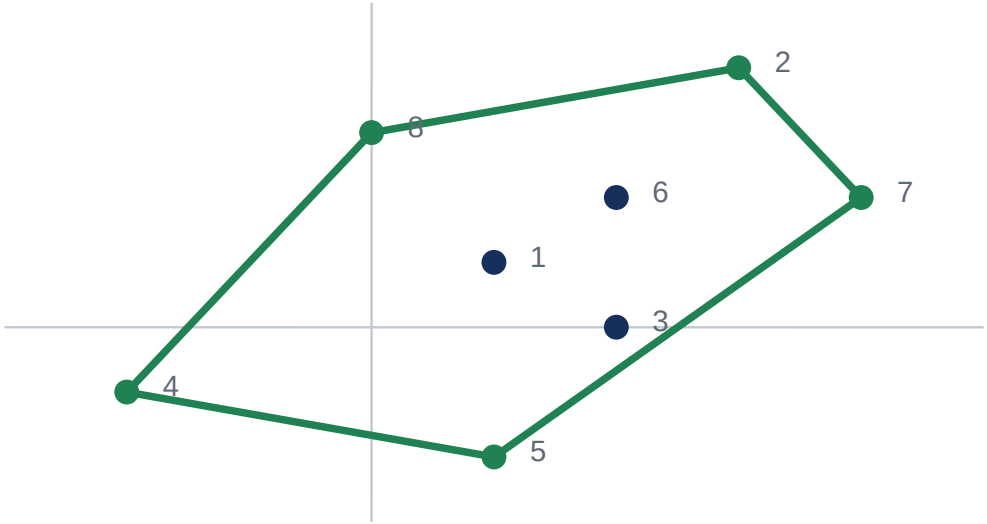
Algoritmos para o fecho convexo

- **Incremental:** insere pontos e atualiza o fecho.
- **Jarvis March (embrulho de presente):** escolhe o próximo vértice.
- **Graham Scan:** ordena angularmente e mantém uma pilha.
- **Monotone Chain (Andrew):** ordena por x e constrói duas cadeias.

Nesta aula, implementaremos o Monotone Chain.

Embrulho de presente - ideia

- Começa em um ponto extremo.
- A cada passo, procura o próximo vértice da fronteira.



Complexidade: O(nh).

h = número de vértices do fecho convexo.

Graham Scan e Monotone Chain

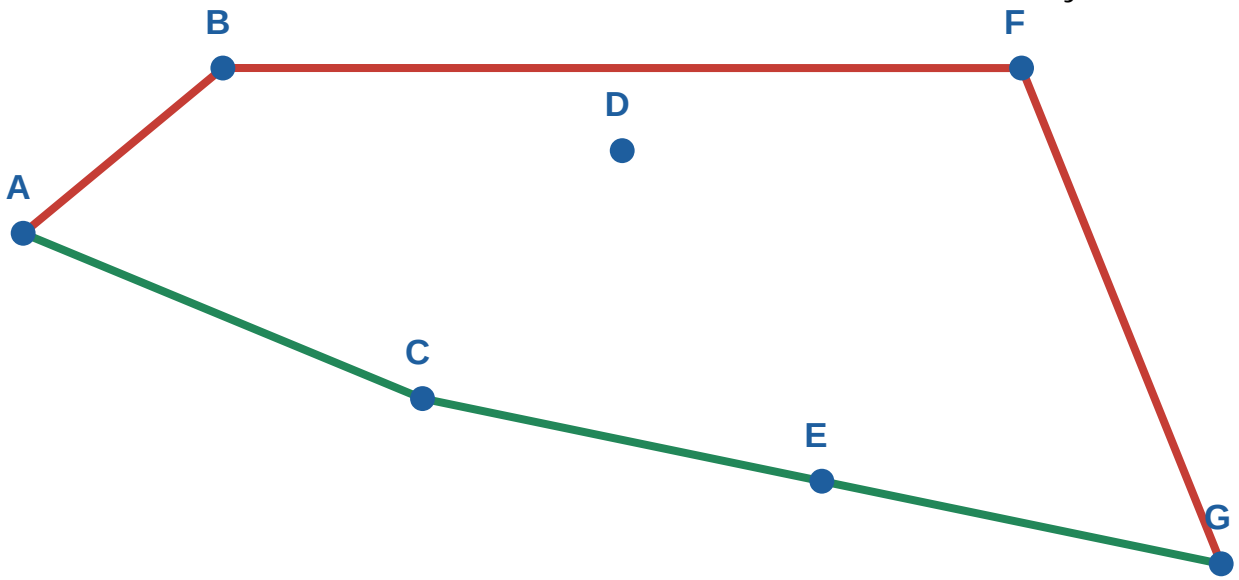
- **Graham Scan:** ordena angularmente em torno de um ponto extremo.
- **Monotone Chain (Andrew):** ordenação lexicográfica (x, depois de y).
- Ambos usam orientação e uma estrutura semelhante a uma pilha.
- Ambos têm complexidade O(n log n).

Monotone Chain – visão geral

- Ordenar os pontos por x (em caso de empate, por y).
- Construir a **cadeia inferior**, removendo curvas inadequadas.
- Construir a **cadeia superior** com a mesma regra.
- **Unir as cadeias** sem repetir os pontos extremos.

Monotone Chain – duas cadeias

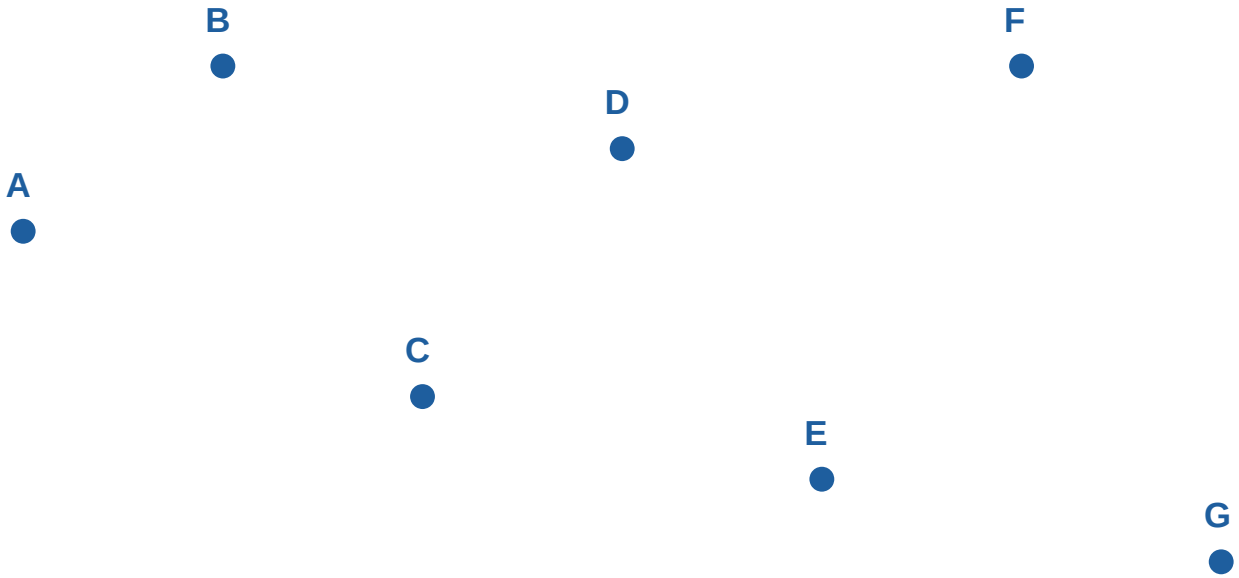
- O fecho é formado por uma cadeia inferior e uma superior.
- As duas serão construídas usando o mesmo teste de orientação.



Verde: inferior **Vermelho: superior**

Monotone Chain – ordenação

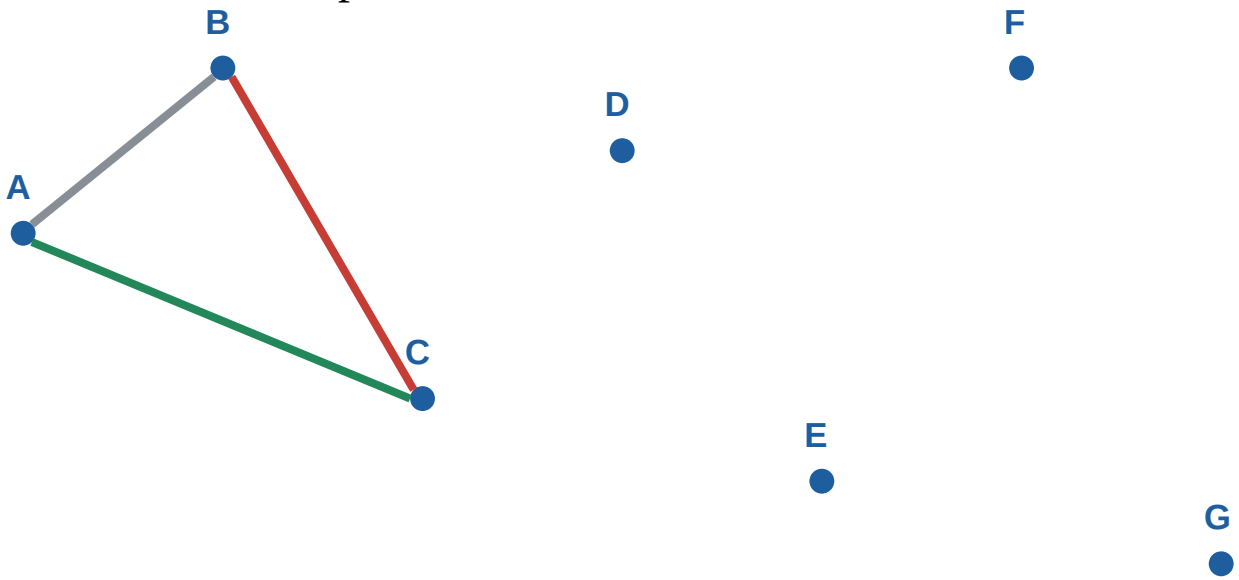
- Primeiro, ordenamos os pontos por x; em caso de empate, por y.
- A ordem da varredura será: A, B, C, D, E, F, G.



A ordem dos pontos determina o percurso do algoritmo.

Monotone Chain – primeiros pontos

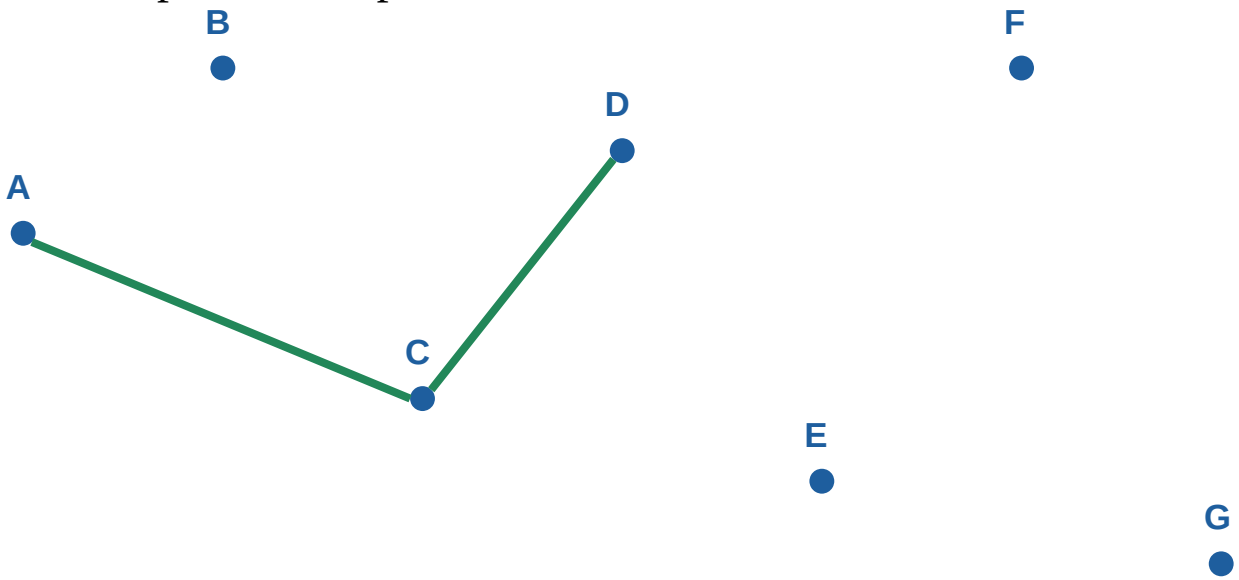
- Começamos inserindo A e B. Ao inserir C, A → B → C faz uma curva à direita.
- Portanto, B não deve permanecer na cadeia inferior.



Remover B; a cadeia inferior passa a ser A → C

Monotone Chain – cadeia inferior – novo candidato

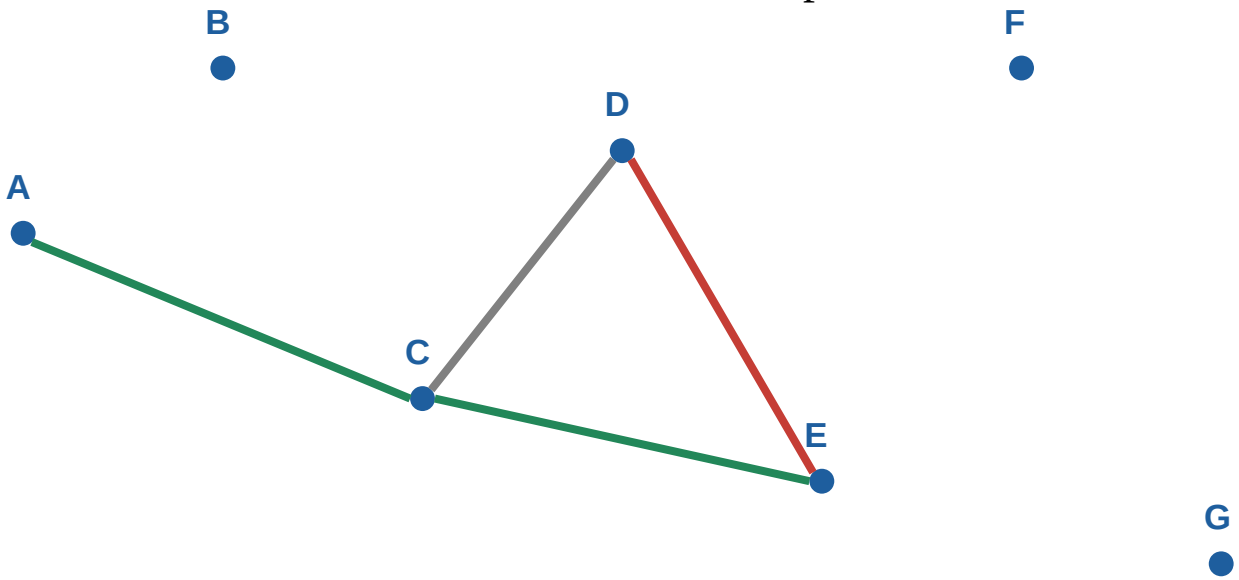
- Depois de A → C, inserimos D. A curva A → C → D é à esquerda.
- Portanto, D permanece provisoriamente na cadeia inferior.



Uma decisão pode mudar quando chegar outro ponto.

Monotone Chain – cadeia inferior – remoção

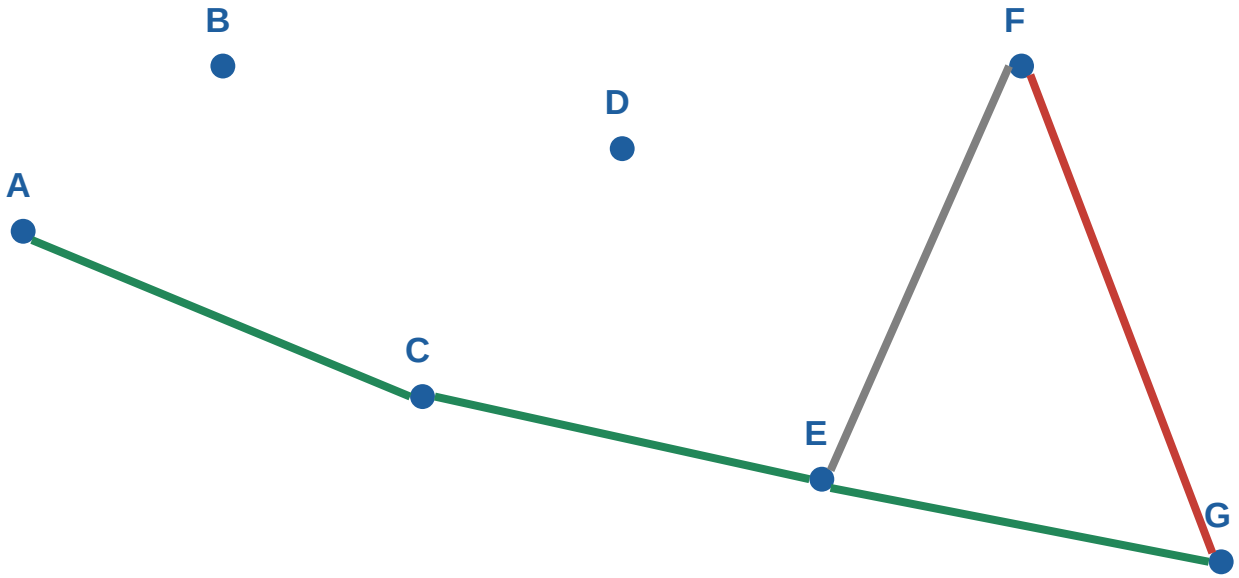
- Ao inserir E, a curva C → D → E fica à direita.
- Removemos D e voltamos a testar os últimos pontos da cadeia.



Após remover D, A → C → E faz curva à esquerda.

Monotone Chain – cadeia inferior – por que *while*?

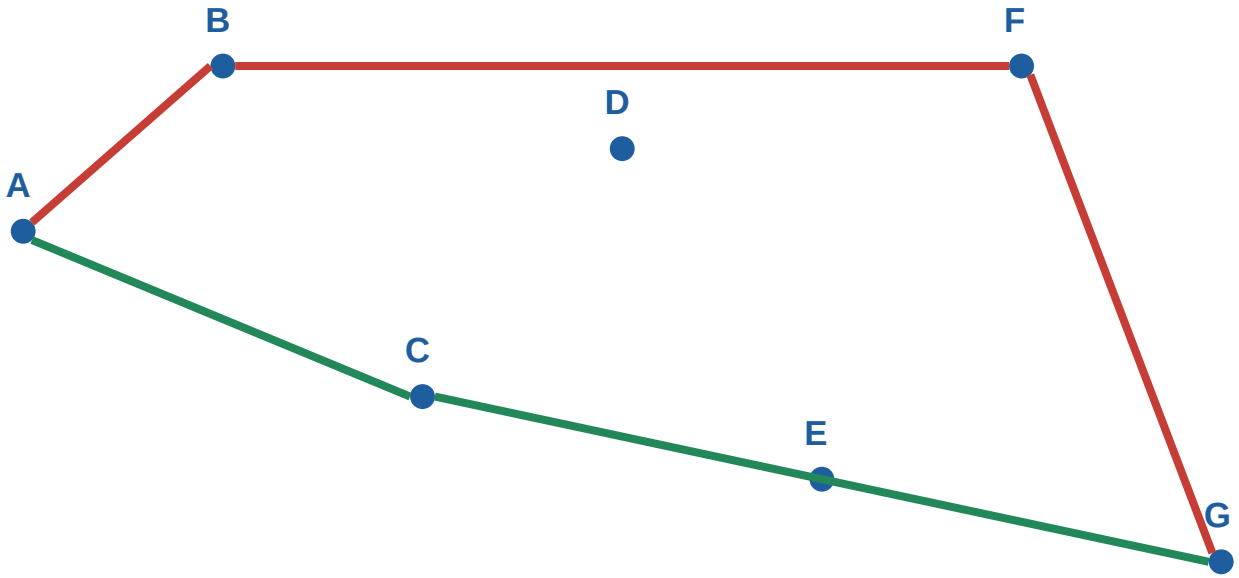
- F é inserido provisoriamente. Quando G é analisado, primeiro removemos F.
- No novo teste C → E → G é **colinear**: removemos E também.



Um único ponto pode provocar várias remoções: usamos *while*.

Monotone Chain – união das cadeias

- Cadeia inferior: A → C → G. Cadeia superior: G → F → B → A.
- Unimos as cadeias sem repetir as extremidades A e G.



Fecho convexo: A → C → G → F → B (sentido anti-horário).

Monotone Chain – estruturas e orientação

```
struct Ponto {
    long long x, y;
};

bool compara(Ponto a, Ponto b) {
    if (a.x != b.x)
       return a.x < b.x;
    return a.y < b.y;
}

// Produto vetorial (B - A) x (C - A).
// > 0: curva para a esquerda; < 0: direita; = 0: colineares.
long long orientacao(Ponto a, Ponto b, Ponto c) {
    return (b.x - a.x) * (c.y - a.y)
         - (b.y - a.y) * (c.x - a.x);
}
```

Monotone Chain – estruturas e orientação

```
vector<Ponto> fechoConvexo(vector<Ponto> pontos) {
    sort(pontos.begin(), pontos.end(), compara);

    // Elimina pontos repetidos.
    pontos.erase(unique(pontos.begin(), pontos.end(),
                        [](Ponto a, Ponto b) {
                            return a.x == b.x && a.y == b.y;
                        }), pontos.end());

    int n = (int)pontos.size();
    if (n <= 1)
       return pontos;

    vector<Ponto> fecho(2 * n);
    int k = 0;
```

Monotone Chain – construção das cadeias

```
// Cadeia inferior: percorre da esquerda para a direita.
for (int i = 0; i < n; i++) {
    while (k >= 2 &&
           orientacao(fecho[k - 2], fecho[k - 1], pontos[i]) <= 0) {
        k--;
    }
    fecho[k++] = pontos[i];
}

// Cadeia superior: percorre da direita para a esquerda.
int limite = k + 1;
for (int i = n - 2; i >= 0; i--) {
    while (k >= limite &&
           orientacao(fecho[k - 2], fecho[k - 1], pontos[i]) <= 0)
        k--;
    fecho[k++] = pontos[i];
}
```

Monotone Chain – construção das cadeias

```
    // O último ponto repete o primeiro.
    fecho.resize(k - 1);
    return fecho;
}
```

Monotone Chain – entrada e saída

```
int main() {
    int n;
    cin >> n;
    vector<Ponto> pontos(n);
    for (int i = 0; i < n; i++) {
        cin >> pontos[i].x >> pontos[i].y;
    }

    vector<Ponto> fecho = fechoConvexo(pontos);

    cout << fecho.size() << '\n';
    for (Ponto p : fecho) {
        cout << p.x << ' ' << p.y << '\n';
    }
    return 0;
}
```

Problemas semelhantes no **beecrowd**

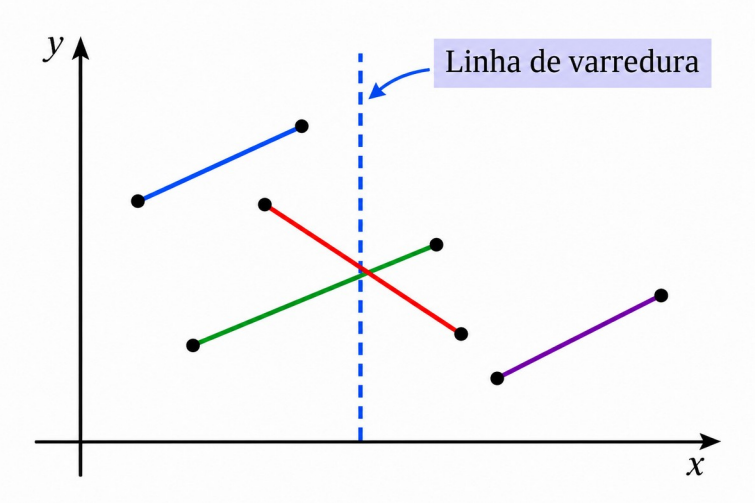


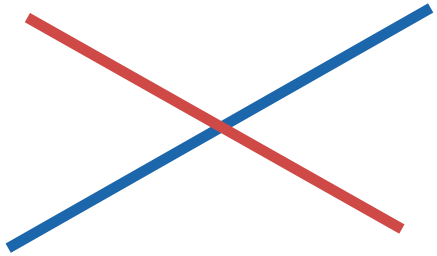




Interseção de segmentos: do simples ao eficiente

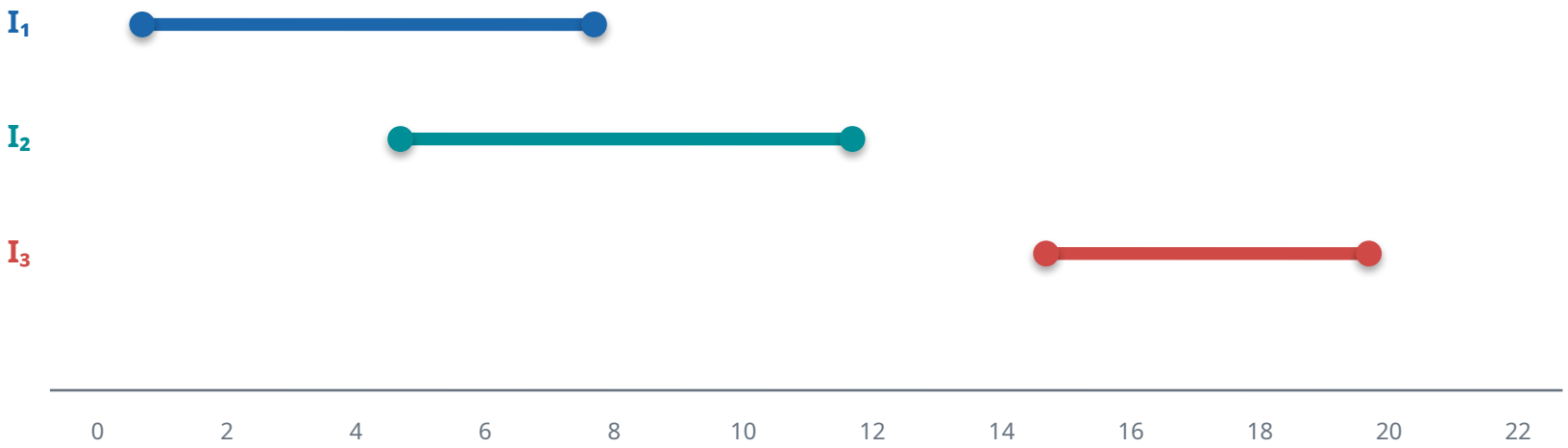
- **Problema:** Dados *n* segmentos, decidir se dois deles se intersectam.
- **Solução direta:** testar todos os pares – $O(n^2)$.
- **Objetivo:** explorar a ordem dos eventos para reduzir o trabalho.





Um problema na reta

- Um segmento na reta é um intervalo [a,b].



I_1 e I_2 se sobrepõem; I_3 está separado.

Ideia: ordenar os extremos dos intervalos

- Cada intervalo gera dois eventos: início e fim.
- Ordenamos os eventos pela coordenada.
- Ao iniciarmos um intervalo, aumentamos a quantidade de ativos.
- Ao terminar um intervalo, diminuímos essa quantidade.

Se ativos ≥ 2, existe interseção.

Atenção aos extremos coincidentes

- Intervalos fechados: [1, 4] e [4, 7] têm o ponto 4 em comum.



Na mesma coordenada, processe INÍCIO antes de FIM.

Significado de varredura

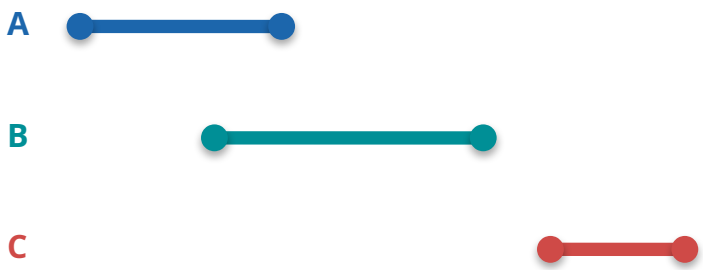
- Imagine um ponto que percorre a reta da esquerda para a direita.
- Ao encontrar o início de um intervalo, ele se torna ativo; ao passar pelo fim, deixa de estar ativo.



A análise muda apenas nos extremos dos intervalos: esses pontos são os eventos.

Simulação da varredura: eventos ordenados

- Intervalos: [0, 3], [2, 6], [7, 9]



Eventos: 0 início A · 2 início B · 3 fim A · 6 fim B · 7 início C · 9 fim C

Simulação da varredura: eventos ordenados

		Ativos
x = 0: inicia A	+1	1
x = 2: inicia B	+1	2
x = 3: termina A	−1	1
x = 6: termina B	−1	0

Em x = 2, ativos = 2 → SIM

Interseção de intervalos – implementação C++

```
struct Evento {
    int x, tipo;    // 0 = início, 1 = fim
};


bool compara(Evento a, Evento b) {
    if (a.x != b.x)
       return a.x < b.x;
    return a.tipo < b.tipo;  // início antes de fim
}
```

Interseção de intervalos – implementação C++

```
bool existeIntersecao(vector<Evento>& eventos) {
    sort(eventos.begin(), eventos.end(), compara);
    int ativos = 0;
    for (Evento e : eventos) {
        if (e.tipo == 0) {
            ativos++;
            if (ativos >= 2)
               return true;
        } else {
            ativos--;
        }
    }
    return false;
}
```

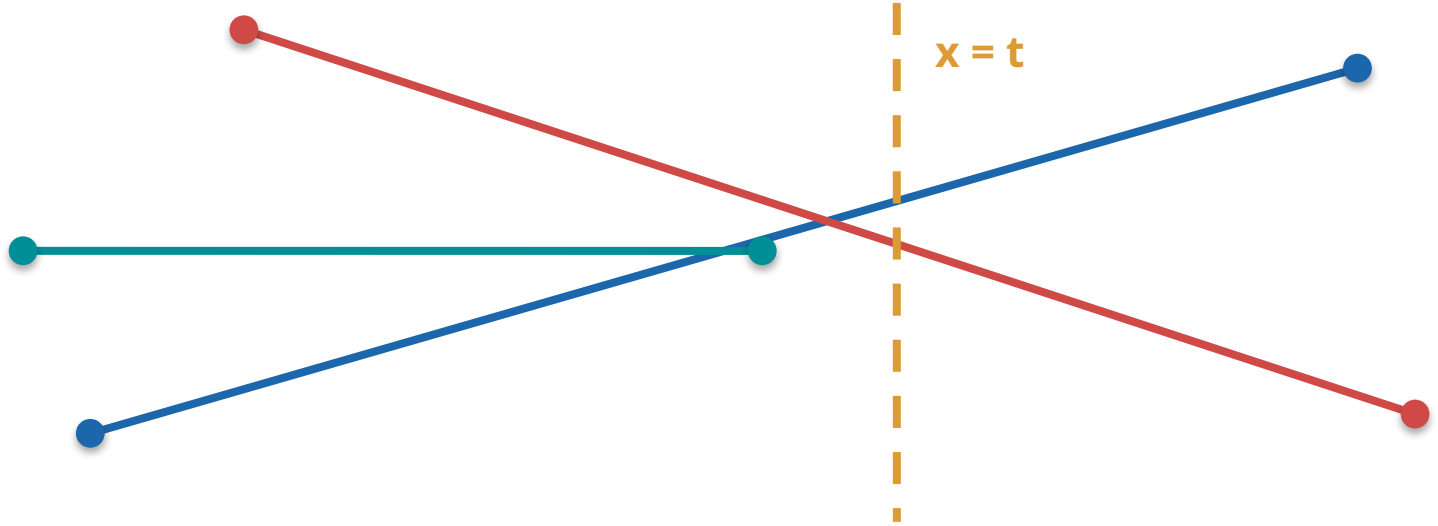
Por que a varredura é eficiente?

- Geramos 2n eventos: O(n).
- Ordenamos os eventos: O(n log n).
- Percorremos a lista uma única vez: O(n).
- Complexidade total: O(n log n).

Da força bruta $O(n^2)$ à varredura O(n log n)

Do intervalo na reta ao segmento no plano

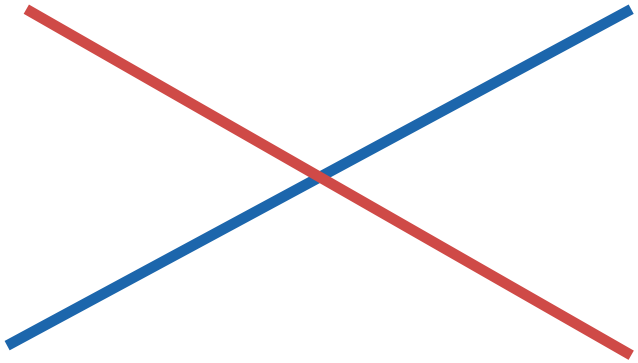
- No plano, a linha vertical x = t percorre os segmentos da esquerda para a direita.



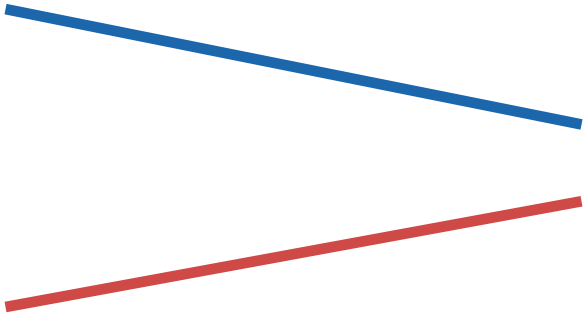
Segmentos ativos: os que cruzam a linha.

Interseção de dois segmentos

- Antes de estudar vários segmentos, precisamos testar apenas dois



Interseção



Sem interseção

Reutilizando o predicado de orientação

- O determinante fornece um teste numérico para a orientação.

$$\mathbf{orient(A,B,C) = (Bx - Ax)(Cy - Ay) - (By - Ay)(Cx - Ax)}$$

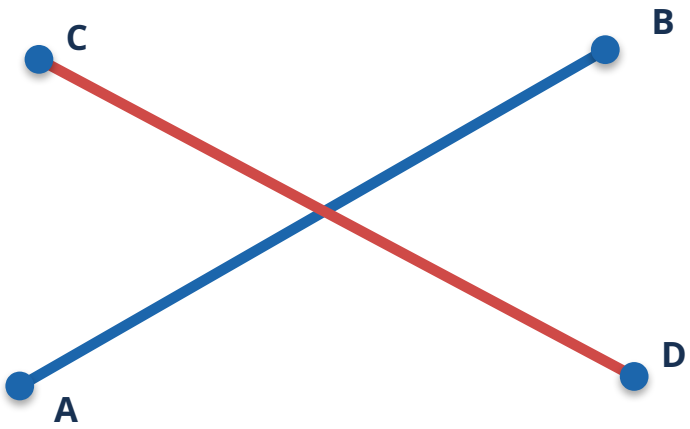
orient(A,B,C) > 0 → C à esquerda de AB

orient(A,B,C) < 0 → C à direita de AB

orient(A,B,C) = 0 → A, B e C são colineares

Reutilizando o predicado de orientação

- Considere os segmentos AB e CD.



C e D em lados opostos da reta AB

A e B em lados opostos da reta CD

Interseção: casos especiais

- Cruzamento próprio: as orientações apresentam sinais opostos.
- Contato em extremidades: também caracteriza uma interseção.
- Pontos colineares: verificar se pertencem aos segmentos.
- Segmentos colineares podem apresentar sobreposição parcial ou total.

Não basta comparar os sinais quando alguma orientação é zero.

Interseção de intervalos – funções auxiliares

```
struct Ponto { long long x, y; };

long long orientacao(Ponto a, Ponto b, Ponto c) {
    return (b.x-a.x)*(c.y-a.y)
         - (b.y-a.y)*(c.x-a.x);
}

bool noSegmento(Ponto a, Ponto b, Ponto p) {
    return min(a.x,b.x) <= p.x && p.x <= max(a.x,b.x)
        && min(a.y,b.y) <= p.y && p.y <= max(a.y,b.y);
}
```

Algoritmo de Shamos e Hoey

- **Problema:** Dados *n* segmentos no plano, determinar se existe pelo menos um par que se intersecta.

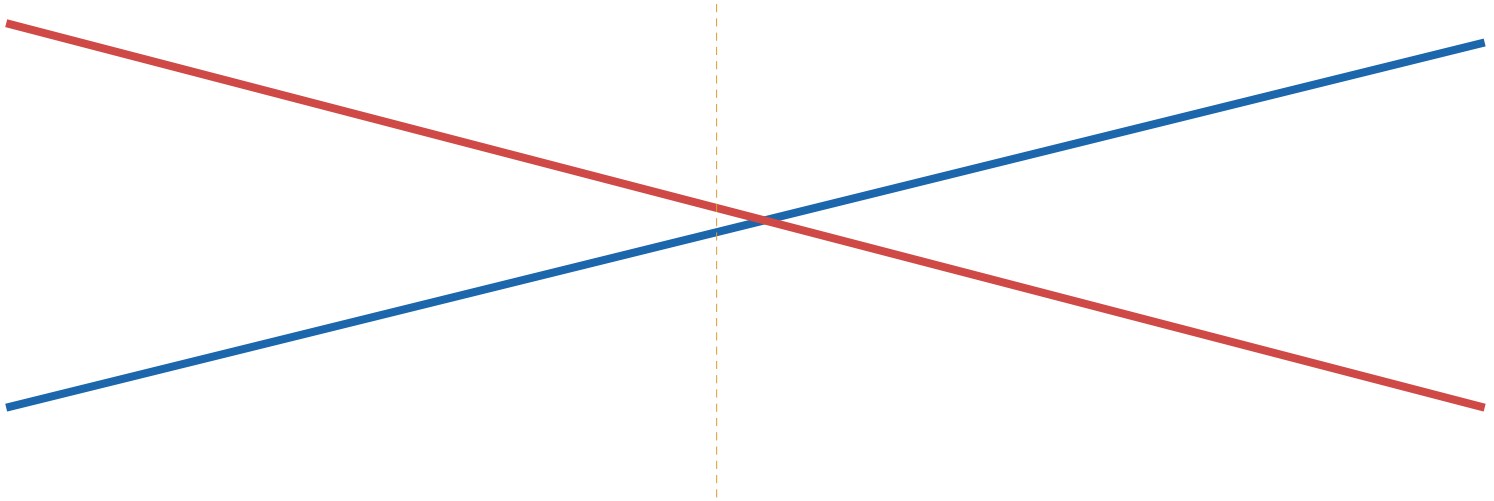
- Varredura: processar os extremos em ordem crescente de x.

- Manter somente os segmentos que cruzam a linha atual.

- Investigar interseções entre vizinhos na ordem vertical.

Por que examinar apenas vizinhos?

- Se dois segmentos se cruzam, sua ordem vertical precisa mudar.



Imediatamente antes de um cruzamento, os dois segmentos são vizinhos.

Interseção de intervalos – teste completo

```
bool intersecta(Ponto a, Ponto b, Ponto c, Ponto d) {
    long long o1 = orientacao(a,b,c);
    long long o2 = orientacao(a,b,d);
    long long o3 = orientacao(c,d,a);
    long long o4 = orientacao(c,d,b);

    if (o1 == 0 && noSegmento(a,b,c)) return true;
    if (o2 == 0 && noSegmento(a,b,d)) return true;
    if (o3 == 0 && noSegmento(c,d,a)) return true;
    if (o4 == 0 && noSegmento(c,d,b)) return true;

    return (o1 > 0) != (o2 > 0) &&
           (o3 > 0) != (o4 > 0);
}
```

Eventos e conjunto ativo

- A linha muda de estado nos extremos dos segmentos.

Início do segmento

1. Inserir no conjunto ativo
2. Testar os dois vizinhos

Fim do segmento

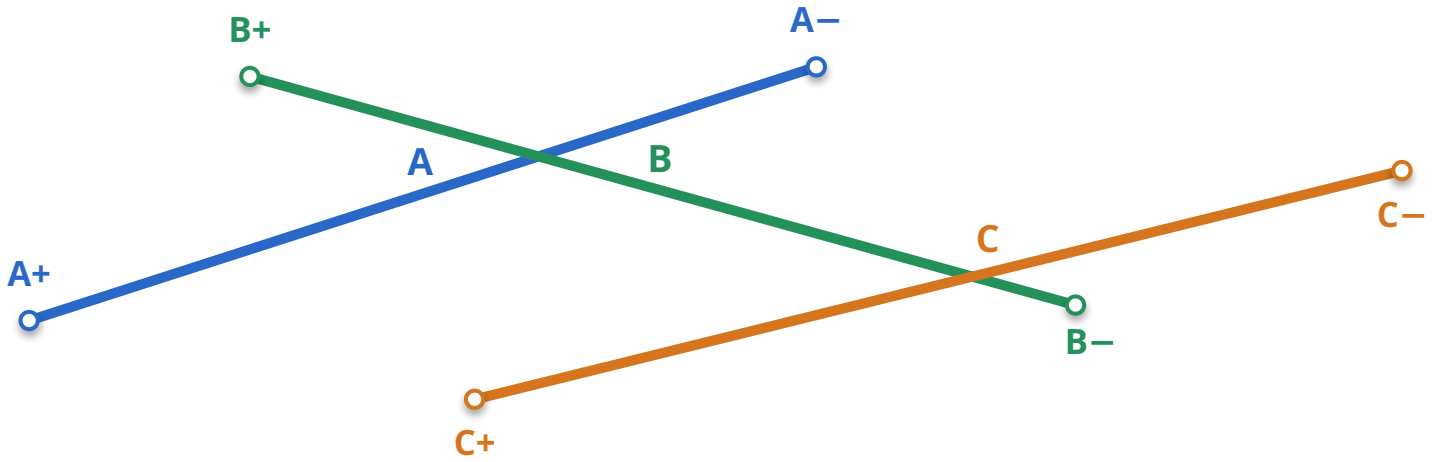
1. Remover do conjunto ativo
2. Testar os ex-vizinhos

Estrutura de dados: ordem vertical

- O conjunto ativo precisa permitir inserção e remoção.
- Também precisamos encontrar predecessor e sucessor.
- Uma **árvore de busca balanceada** realiza essas operações em O(log n).
- Em C++, `std::set` é uma opção, mas o comparador geométrico exige cuidado.

Fila de eventos

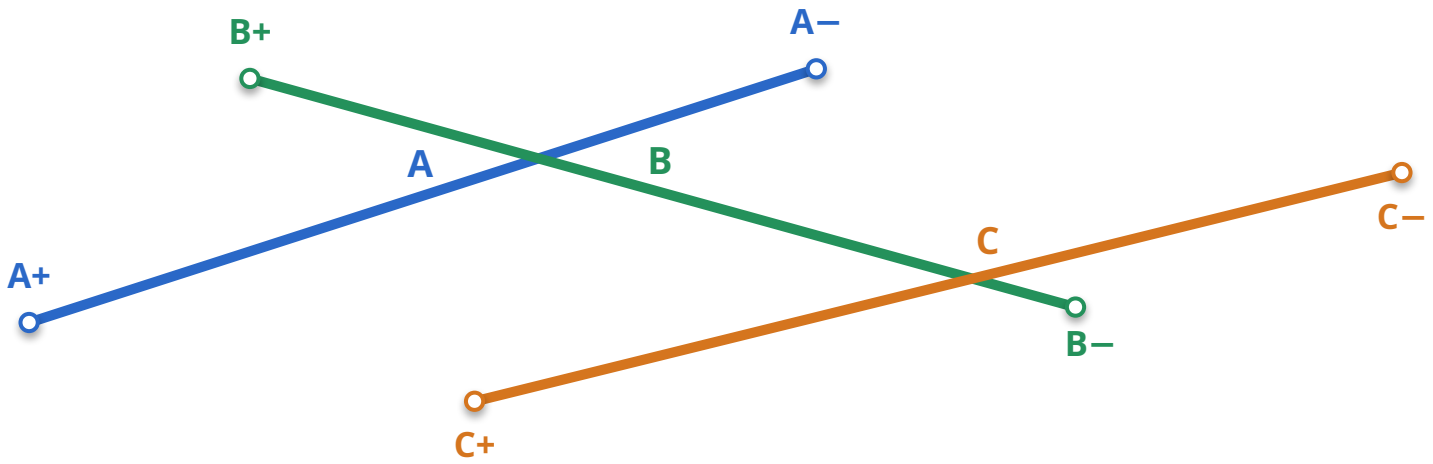
- Cada segmento possui duas extremidades que serão processadas pela varredura.



Cada segmento gera dois eventos: inserção na extremidade esquerda e remoção na direita.

Fila de eventos

- Os extremos são ordenados pela coordenada x, da esquerda para a direita.



A+ → B+ → C+ → A− → B− → C−

Fila de eventos

- Ordenar os 2n extremos pela coordenada x.
- Evento de início: inserir o segmento no conjunto ativo.
- Evento de fim: retirar o segmento do conjunto ativo.
- Empates e segmentos verticais exigem tratamento explícito.

Conjunto ativo T

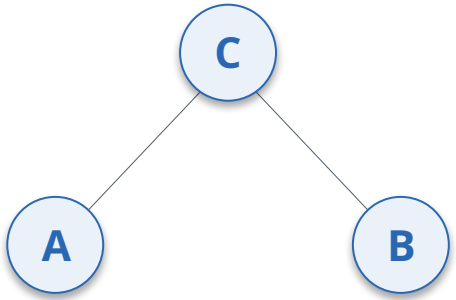
- A linha vertical x=t define quais segmentos estão ativos.
- Um segmento está ativo enquanto a varredura percorre sua projeção no eixo x.
- Os ativos são ordenados pela altura em que cruzam x=t.
- Precisamos das operações: inserir, remover, predecessor e sucessor.

Representação do conjunto ativo em uma ABB

- Uma árvore de busca balanceada mantém a ordem vertical dos segmentos.

Ordem vertical:

A (inferior) < C < B (superior)



A ABB representa a ordem, não as coordenadas dos extremos.

Simulação – segmentos e eventos

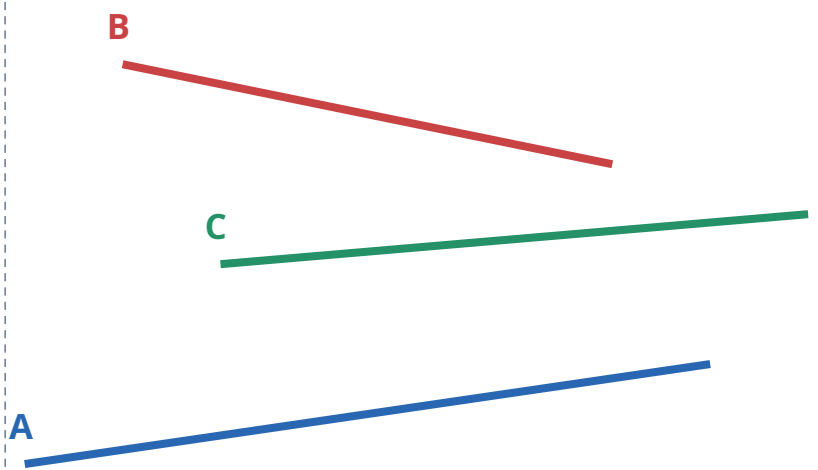
- A, B e C não se intersectam: veremos como a estrutura é atualizada.

ABB: ordem vertical

T = ∅

Fila de eventos (restantes)

1 A+	2 B+	3 C+
4 B–	5 A–	6 C–



Simulação – segmentos e eventos

- Exemplo sem interseções: A = (1,1)–(8,2), B = (2,5)–(7,4), C = (3,3)–(9,3).

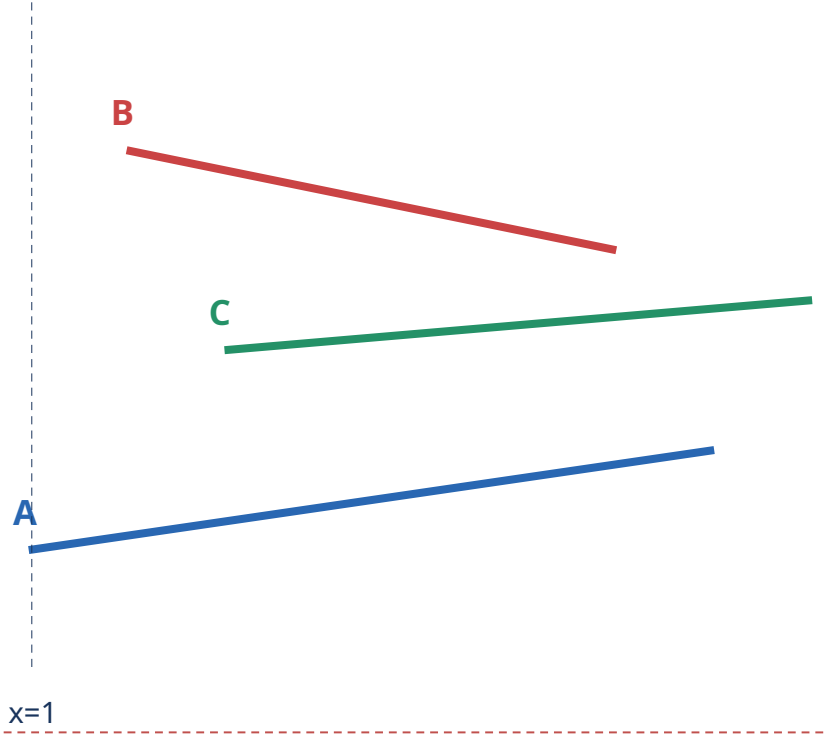
Inserir A em T.

ABB: ordem vertical

A

Fila de eventos (restantes)

2 B+	3 C+	4 B–
5 A–	6 C–	



Simulação – segmentos e eventos

- Exemplo sem interseções: A = (1,1)–(8,2), B = (2,5)–(7,4), C = (3,3)–(9,3).

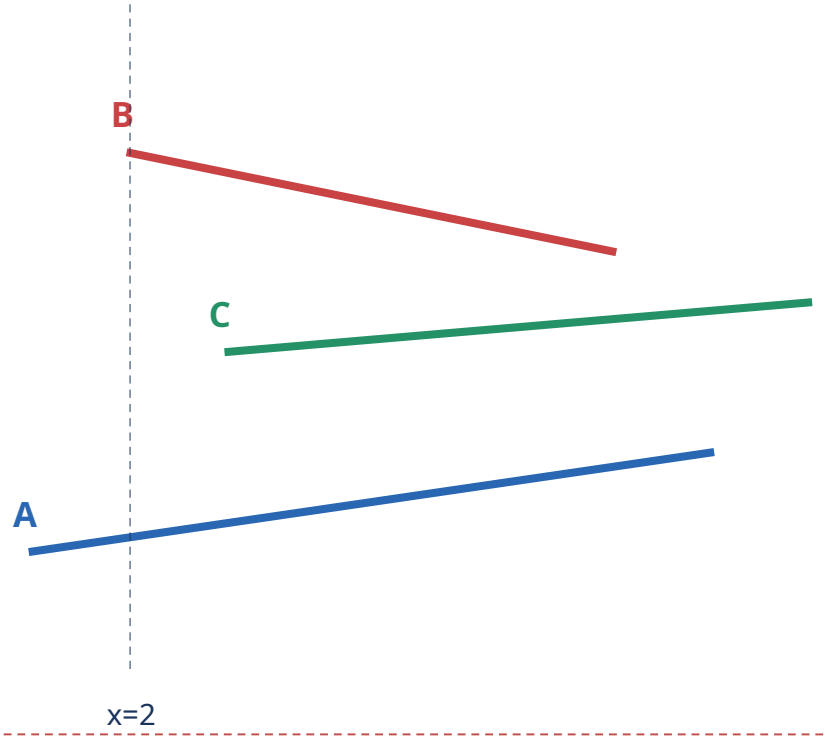
Inserir B em T.

ABB: ordem vertical

A < B

Fila de eventos (restantes)

3 C+ | 4 B– | 5 A– | 6 C–



Simulação – segmentos e eventos

- Exemplo sem interseções: A = (1,1)–(8,2), B = (2,5)–(7,4), C = (3,3)–(9,3).

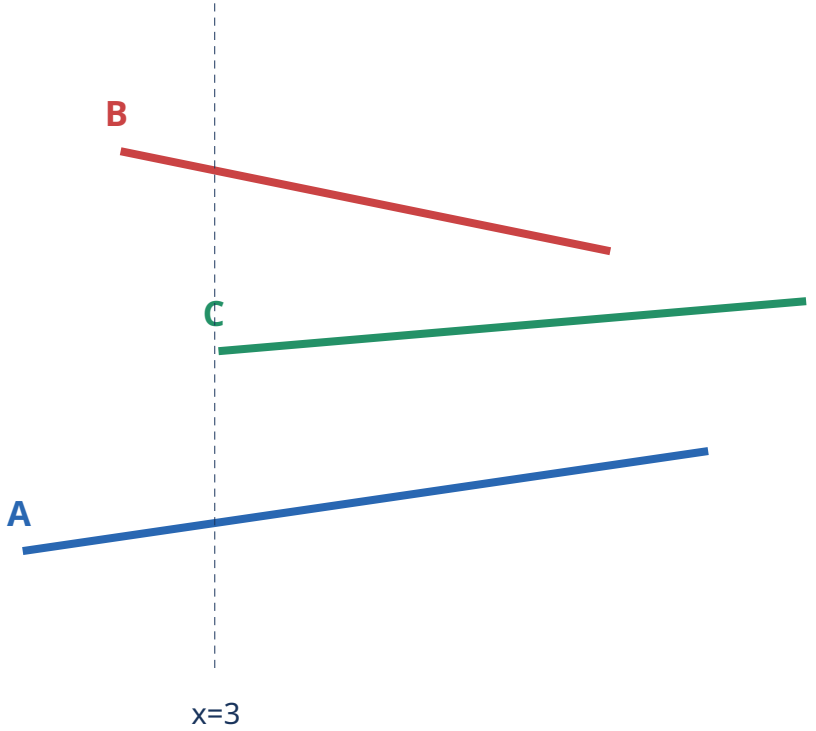
Inserir C em T (entre A e B).

ABB: ordem vertical

A < C < B

Fila de eventos (restantes)

4 B–	5 A–	6 C–



Simulação – segmentos e eventos

- Exemplo sem interseções: A = (1,1)–(8,2), B = (2,5)–(7,4), C = (3,3)–(9,3).

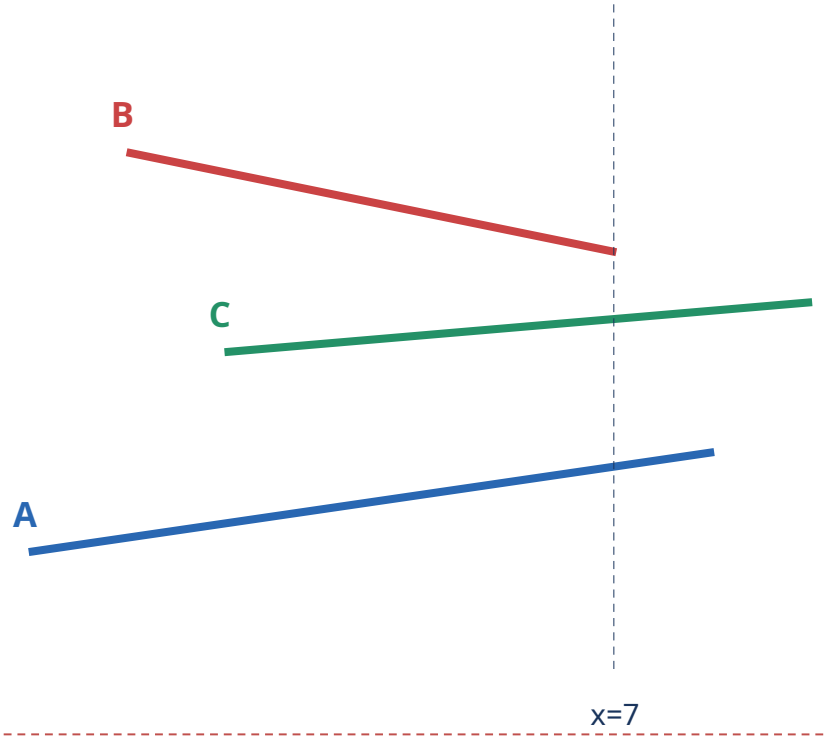
Remover B de T.

ABB: ordem vertical

A < C

Fila de eventos (restantes)

5 A–	6 C–



Simulação – segmentos e eventos

- Exemplo sem interseções: A = (1,1)–(8,2), B = (2,5)–(7,4), C = (3,3)–(9,3).

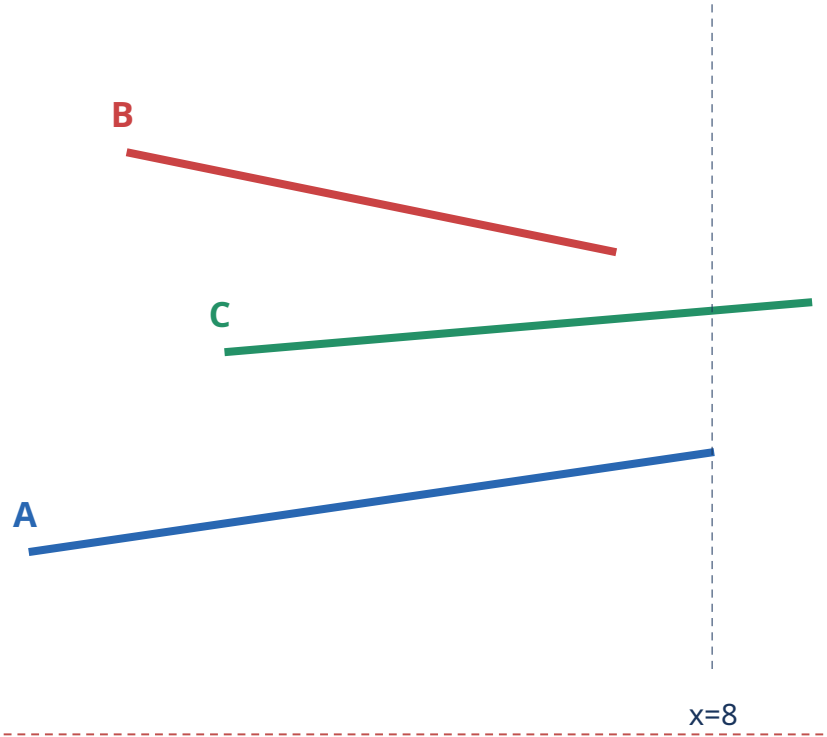
Remover A de T.

ABB: ordem vertical

C

Fila de eventos (restantes)

6 C–



Simulação – segmentos e eventos

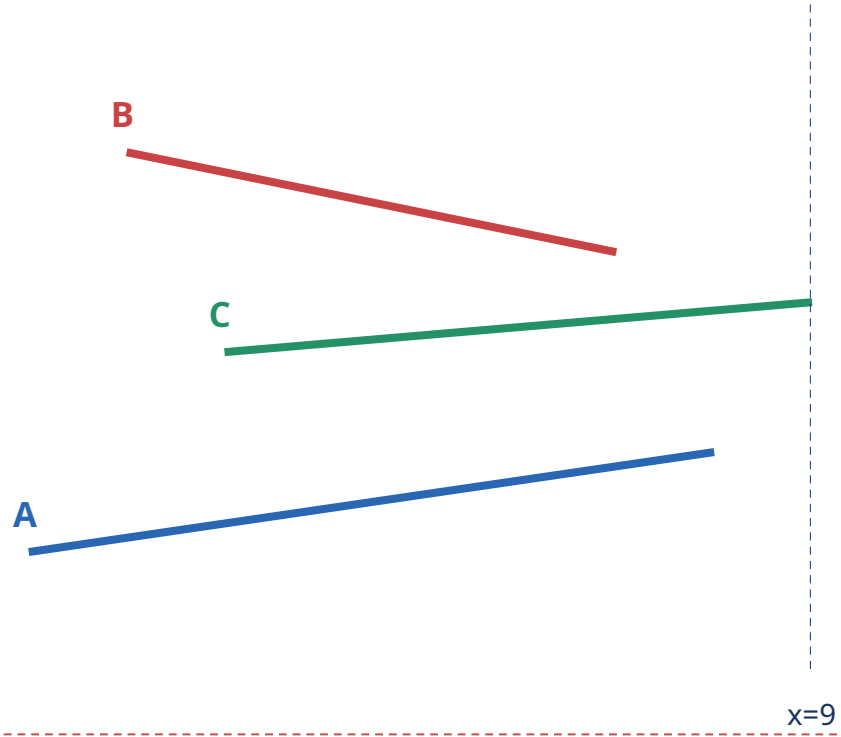
- Exemplo sem interseções: A = (1,1)–(8,2), B = (2,5)–(7,4), C = (3,3)–(9,3).

Remover C de T.

ABB: ordem vertical

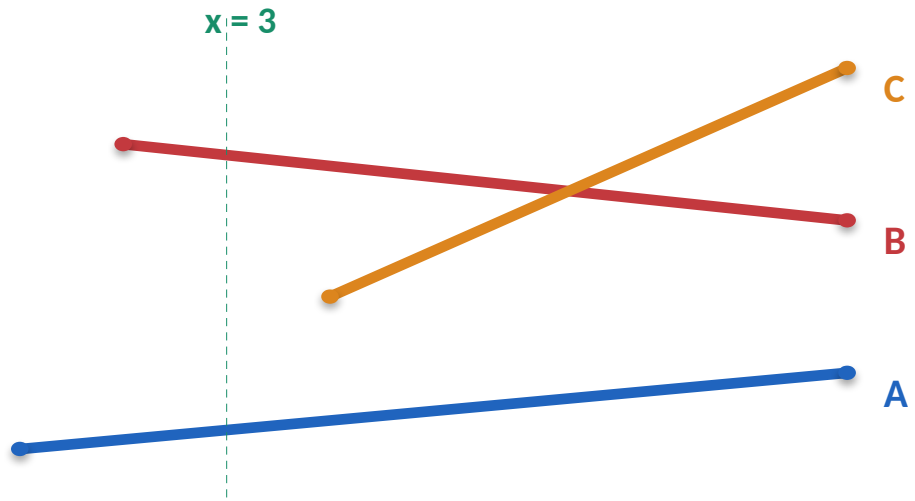
T = ∅

Fila de eventos (restantes)



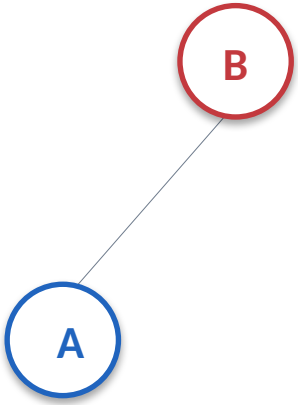
Detecção de interseção

- Antes do evento C+: a linha de varredura está em x=3.



Segmentos no plano

ABB (ordem vertical)

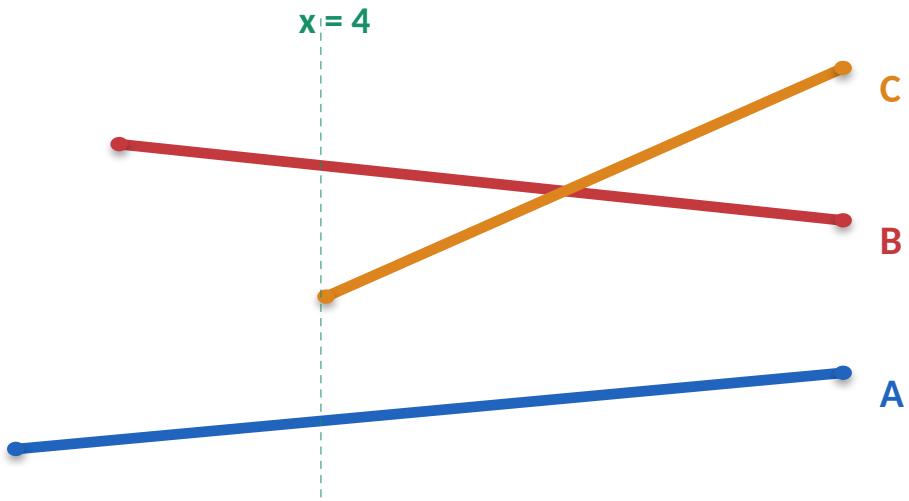


Ativos: A e B

A está abaixo de B. O segmento C ainda não começou.

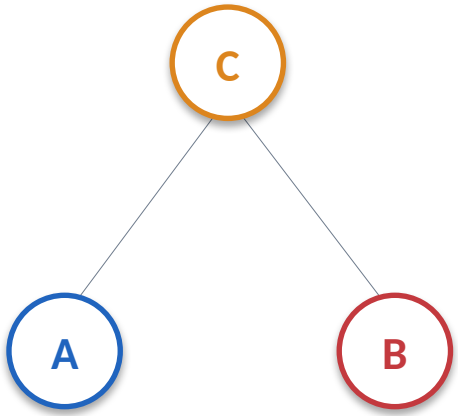
Detecção de interseção

- Ao alcançar a extremidade esquerda de C, o segmento torna-se ativo.



Segmentos no plano

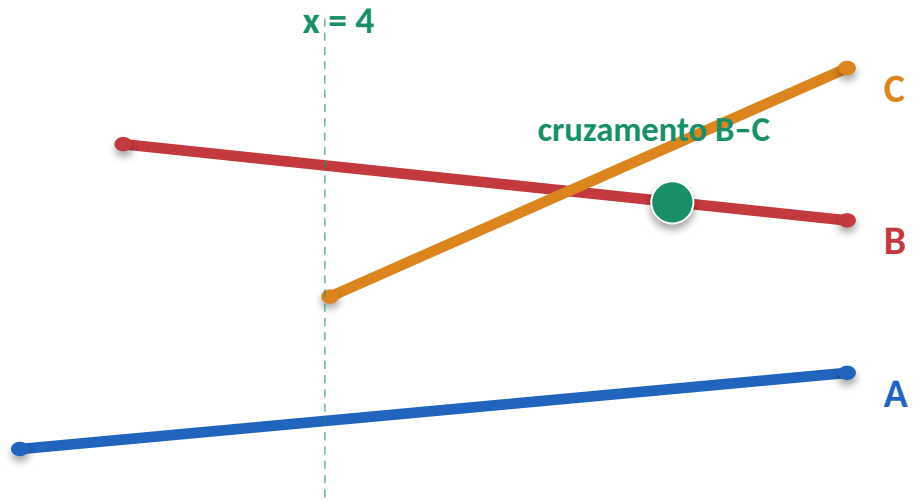
ABB após inserir C



Vizinhos imediatos de C:
A (abaixo) e B (acima).

Detecção de interseção

- Após inserir C, o algoritmo testa C-A e C-B.



Segmentos no plano

Teste após a inserção

C × A: não intersectam

C × B: intersectam

Resultado: SIM

O teste geométrico identifica o cruzamento à direita da linha

Shamos e Hoey - pseudocódigo

A fila E está ordenada; T mantém os segmentos ativos.

```
E ← extremos dos segmentos ordenados por x
T ← ABB vazia

para cada evento e em E:
    se e é início de s:
        inserir(T, s)
        se intersecta(s, predecessor(T,s)): retornar SIM
        se intersecta(s, sucessor(T,s)): retornar SIM
```

Shamos e Hoey - pseudocódigo

Antes de remover s, seus dois vizinhos podem tornar-se adjacentes.

```
        senão:  // e é fim de s
            a ← predecessor(T,s)
            b ← sucessor(T,s)
            remover(T,s)
            se intersecta(a,b): retornar SIM

    retornar NÃO
```

Complexidade e cuidados de implementação

- Ordenação dos 2n eventos: O(n log n).
- Cada evento faz um número constante de operações na árvore: O(log n).
- Complexidade: O(n log n), sob hipóteses de tratamento correto de degenerações.
- Extremos com o mesmo x, segmentos verticais e sobreposições exigem regras específicas.

Shamos e Hoey – estruturas e eventos

```
struct Ponto {
    long long x, y;
};

struct Segmento {
    Ponto a, b;
};

struct Evento {
    long long x;
    int id, tipo;
};
// tipo = +1: inicio; tipo = -1: fim

vector<Segmento> seg;
long long xAtual;
```

Shamos e Hoey – teste geométrico de interseção

```
long long orientacao(Ponto a, Ponto b, Ponto c) {
    return (b.x-a.x)*(c.y-a.y) - (b.y-a.y)*(c.x-a.x);
}

bool intersectam(int i, int j) {
    auto [a,b] = seg[i];
    auto [c,d] = seg[j];

    long long p = orientacao(a,b,c);
    long long q = orientacao(a,b,d);
    long long r = orientacao(c,d,a);
    long long s = orientacao(c,d,b);

    return ((p < 0 && q > 0) || (p > 0 && q < 0)) &&
           ((r < 0 && s > 0) || (r > 0 && s < 0));
}
```

Shamos e Hoey – ABB: ordem vertical em xAtual

```
struct Compara {
    bool operator()(int i, int j) const {
        if (i == j) return false;

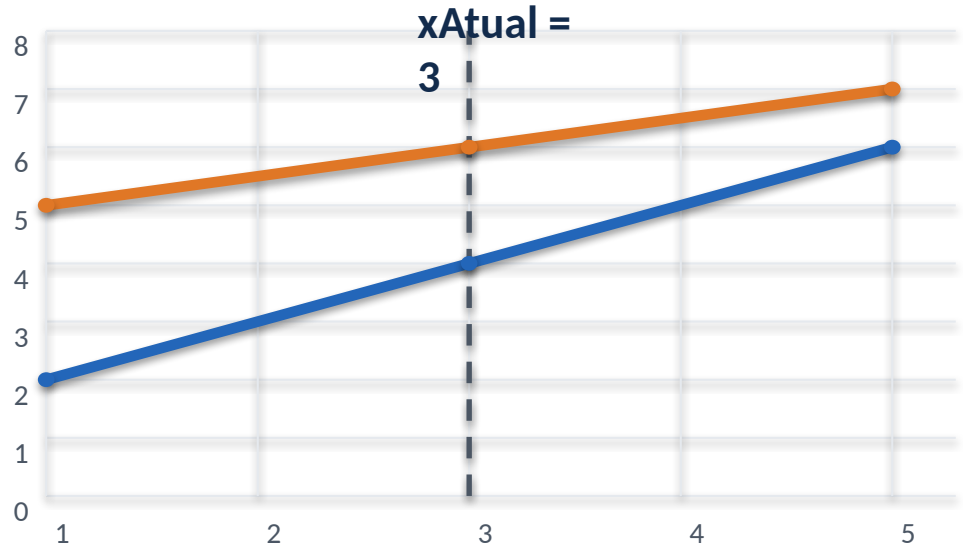
        const auto &u = seg[i], &v = seg[j];

        long long du = u.b.x - u.a.x;
        long long dv = v.b.x - v.a.x;
        long long nu = u.a.y * du +
                       (u.b.y - u.a.y) * (xAtual - u.a.x);
        long long nv = v.a.y * dv +
                       (v.b.y - v.a.y) * (xAtual - v.a.x);

        if (nu * dv != nv * du)
            return nu * dv < nv * du;
        return i < j;
    }
};
```

Compara: qual segmento está mais abaixo?

- A comparação depende da posição atual da linha de varredura.



u: A(1,2) → B(5,6)

v: C(1,5) → D(5,7)

Em x = 3:

$y_u = 4$ e $y_v = 6$

Logo, u está abaixo de v.

Compara: como a função calcula a altura?

- A função evita divisão e compara duas frações por multiplicação cruzada.
- Altura de um segmento na posição xAtual.

$$y = y_a + (y_\beta - y_a) \cdot (xAtual - x_a) / (x_\beta - x_a)$$

$$\mathbf{d = x_\beta - x_a}$$

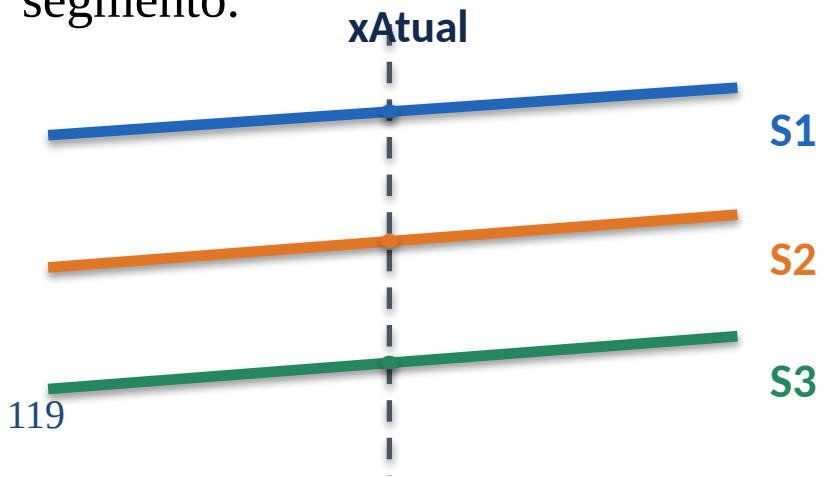
$$\mathbf{n = y_a \cdot d + (y_\beta - y_a) \cdot (xAtual - x_a)}$$

Assim: y = n / d

u abaixo de v ⇔ nu / du < nv / dv ⇔ nu · dv < nv · du

Compara: é usado no <set>.

- O conjunto de ativos precisa estar ordenado por altura, não pelo índice do segmento.



set<int, Compara> ativos;

S1 — acima
S2 — atual
S3 — abaixo

next(it) → vizinho acima
prev(it) → vizinho abaixo

Compara(i, j) retorna true quando i está abaixo de j.

Shamos e Hoey – Fila de eventos

```
bool existeIntersecao() {
    vector<Evento> ev;
    for (int i=0; i<(int)seg.size(); i++) {
        ev.push_back({seg[i].a.x, i, +1});
        ev.push_back({seg[i].b.x, i, -1});
    }
    sort(ev.begin(), ev.end(), [](Evento a, Evento b) {
        return a.x < b.x;
    });

    set<int, Compara> ativos;
    for (Evento e : ev) {
        xAtual = e.x;
        if (e.tipo == +1) {
            auto it = ativos.insert(e.id).first;
            auto prox = next(it);
            if (it != ativos.begin() && intersectam(*prev(it), *it))
                return true;
            if (prox != ativos.end() && intersectam(*it, *prox))
                return true;
```

Shamos e Hoey – Fila de eventos

```
        } else {
            auto it = ativos.find(e.id);
            auto prox = next(it);
            if (it != ativos.begin() && prox != ativos.end() &&
                intersectam(*prev(it), *prox)) return true;
            ativos.erase(it);
        }
    }
    return false;
}
```

Shamos e Hoey – Programa principal

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n; cin >> n;
    seg.resize(n);
    for (auto &s : seg) {
        cin >> s.a.x >> s.a.y >> s.b.x >> s.b.y;
        if (s.a.x > s.b.x) swap(s.a, s.b);
    }
    cout << (existeIntersecao() ? "SIM" : "NAO") << '\n';
    return 0;
}
```

Observação importante:

Comparações com double: por que usar uma tolerância (ε)?

- O tipo **double** representa números reais com precisão finita (≈ 15–16 dígitos decimais).
- Operações aritméticas podem introduzir pequenos erros de arredondamento.
- Comparar valores diretamente com `==` (por exemplo, testar se um valor é zero) pode produzir resultados incorretos.

⚠ **Consequência:** um ponto que deveria ser considerado colinear pode ser classificado como à esquerda ou à direita.

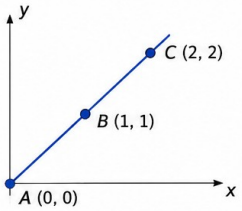
Exemplo: produto vetorial (orientação)

Considere os pontos
$A = (0, 0)$, $B = (1, 1)$,
$C = (2, 2)$.

Geometricamente, A, B e C são colineares, logo o valor da orientação é 0.

Com **double**, porém, podemos obter, por exemplo:

orientacao(A, B, C) = −2.220446e−16



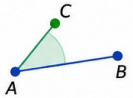
Como usar a tolerância (ε) nas comparações

- Utilize uma tolerância ε (também chamada de EPS) para comparar valores reais com zero.
- Um ponto será considerado colinear se o valor absoluto da orientação for menor que ε.

```
const double EPS = 1e-9;  // tolerância

int orientacao(Ponto a, Ponto b, Ponto c) {
   double v = (b.x - a.x) * (c.y - a.y) -
              (b.y - a.y) * (c.x - a.x);
   if (fabs(v) < EPS) return 0;  // colinear
   return (v > 0) ? +1 : -1;     // esquerda / direita
}
```

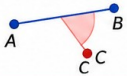
Interpretação geométrica



orientacao > ε (à esquerda)



|orientacao| ≤ ε (colinear)



orientacao < $-\varepsilon$ (à direita)

ⓘ **Observação:** o valor de ε depende da escala dos dados do problema (por exemplo, 1e−9, 1e−8 ou 1e−6).

Problemas semelhantes no **beecrowd**



Aula 11:
Geometria Computacional

***Disciplina*: Maratona de Programação I**

Profs. Edmilson Marmo e Luiz Olmes

edmarmo@unifei.edu.br, olmes@unifei.edu.br

UNIFEI
UNIVERSIDADE FEDERAL DE ITAJUBÁ